TUGAS BESAR IF3170 INTELEGENSI ARTIFISIAL: PROGRAM PENCARI KONFIGURASI PENGISIAN PAKET PALING OPTIMAL UNTUK SEBUAH TRUK PETI KEMAS



Disusun oleh:

Edward David Rumahorbo	13524036
Miguel Rangga Deardo Sinaga	13524069
Reysha Syafitri Mulya Ramadhan	13524137

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

BAB I DESKRIPSI PERSOALAN....4
1.1. Latar Belakang.... 4
1.2 Variabel Permasalahan.... 4
1.3. Ketentuan Representasi....5
1.4. Objective Function dan Constraint....5
1.5. Persoalan Uji.... 5
BAB II PEMBAHASAN.... 7
2.1. Representasi State dan Sistem Koordinat....7
2.2. Objective Function dan Penanganan Constraint.... 8
2.3 Neighbor Moves dan Initial State....11
2.4 Kerangka Algoritma....17
2.5 Hill-climbing....21
2.6 Simulated Annealing....26
2.7 Genetic Algorithm....27
2.8 Bonus 1: Objective Function Alternatif.... 30
2.9 Bonus 3: Multi-truk....31
2.10 Program Visualisasi....31
Gambar 1 Tampilan program visualisasi (Sideways Move Hill-climbing, seed 1).....32
2.11 Hasil Eksperimen Hill-climbing....32
2.12 Hasil Eksperimen Simulated Annealing.... 40
2.13 Hasil Eksperimen Genetic Algorithm.... 42
2.14 Hasil Eksperimen Bonus.... 47
2.15 Analisis Perbandingan Antaralgoritma.... 51
BAB III KESIMPULAN DAN SARAN....54
3.1 Kesimpulan....54
3.2 Saran....54
BAB IV PEMBAGIAN TUGAS.... 56
REFERENSI....57
LAMPIRAN A FORM PENGGUNAAN GENERATIVE AI....58

DAFTAR TABEL

Tabel 1 Variabel permasalahan....4
Tabel 2 Persoalan uji....6
Tabel 3 Contoh representasi penempatan paket....7
Tabel 4 Fungsi dan kelas pada representasi....7
Tabel 5 Jenis pelanggaran constraint....9
Tabel 6 Validasi state setelah satu langkah dari state valid (medium, 40 state awal acak)....13
Tabel 7 Fungsi dan kelas pada neighbor moves dan initial state....14
Tabel 8 Isi hasil run (RunResult)....18
Tabel 9 Fungsi dan kelas pada kerangka algoritma....19
Tabel 10 Varian Hill-climbing dan parameter yang dipakai....22
Tabel 11 Parameter Simulated Annealing....27
Tabel 12 Parameter Genetic Algorithm....28
Tabel 13 Ringkasan hasil Hill-climbing (rata-rata tiga run)....33
Tabel 14 Hasil Hill-climbing per run....33
Tabel 15 Sideways Move Hill-climbing: pemakaian sideways move....34
Tabel 16 Random Restart Hill-climbing: restart dan iterasi per restart....34
Tabel 17 Hasil Simulated Annealing per run....40
Tabel 18 Variasi banyak iterasi (populasi 30)....43
Tabel 19 Variasi jumlah populasi (iterasi 40)....43
Tabel 20 Hasil Genetic Algorithm per run....43
Tabel 21 Total value akhir pada persoalan medium: objective bawaan dan alternatif (3 run)....47
Tabel 22 Rata-rata ETA paket yang diangkut pada state akhir (medium, 3 run)....47
Tabel 23 Total value akhir pada persoalan multi_truck (objective bawaan, rata-rata tiga run)....49
Tabel 24 Batas atas dan nilai terbaik yang ditemukan....51
Tabel 25 Perbandingan algoritma pada persoalan medium (objective bawaan, tiga run)....51

BAB I DESKRIPSI PERSOALAN

1.1. Latar Belakang

Keluarga Mejiro berencan

a mengekspansi bisnisnya ke bidang logistik darat dengan melakukan uji coba berskala kecil bernama *Mejiro Express*. Seorang pengemudi truk uji coba membutuhkan pengaturan pengisian paket yang optimal, yaitu susunan paket di dalam peti kemas truk yang menghasilkan nilai angkut setinggi mungkin tanpa melanggar batasan fisik. Persoalan ini merupakan variasi dari masalah pengepakan tiga dimensi (*3D bin packing*) yang bersifat kombinatorial sehingga tidak praktis diselesaikan secara eksak, dan karena itu dikerjakan dengan algoritma *local search*.

Tujuan tugas ini adalah merancang representasi solusi, mengimplementasikan *Hill-climbing*, *Simulated Annealing*, dan *Genetic Algorithm*, menjalankan eksperimen, serta menganalisis hasilnya.

1.2 Variabel Permasalahan

Tabel 1 Variabel permasalahan

Entitas	Atribut	Keterangan
Package	id	*Identifier* paket
	Dimensions	Ukuran paket dalam bilangan bulat
	Orientation	Sumbu yang sejajar dengan tiap sisi paket, ada 6 kemungkinan orientasi
	Value	Nilai harga paket
	Weight	Berat paket
	IsFragile	Penanda barang yang pecah-belah
	ETA	Perkiraan waktu sampai

		paket
Truck	Dimensions	Ukuran peti kemas
	MaxCapacity	Kapasitas angkut maksimum

1.3. Ketentuan Representasi

Terdapat beberapa ketentuan yang kami gunakan dalam penyusunan program ini:

- Paket berbentuk kubus atau balok dengan ukuran yang dapat berbeda satu sama lain dan tidak semua paket harus dimasukkan ke dalam peti kemas. Paket di luar berarti seluruh bagiannya berada di luar peti kemas.
- Koordinat berupa bilangan bulat dan cara representasinya dibebaskan.
- *Initial state* dibangkitkan secara acak.
- Langkah *neighbor* yang diperbolehkan yaitu menukar dua paket (di dalam maupun di luar truk), memindahkan paket ke koordinat lain, dan merotasi paket 90 derajat terhadap sumbu x, y, atau z.

1.4. Objective Function dan Constraint

Objective function dari program ini adalah memaksimalkan total *value* paket yang berada di dalam peti kemas. Solusi dikatakan valid jika memenuhi *constraint* berikut:

1. Paket tidak boleh melayang dan tidak boleh menembus dinding peti kemas. Paket dikatakan melayang jika tidak ada satu pun titik alasnya yang tersangga, yaitu berada di lantai truk (z = 0) atau tepat di atas sisi atas paket lain.
2. Barang pecah-belah tidak boleh berada di bawah paket lain, artinya tidak boleh menjadi penyangga.
3. Total berat paket dalam truk tidak boleh melebihi kapasitasnya.

Pada spesifikasi bonus, program diperluas untuk menerima lebih dari satu truk dan memaksimalkan total value yang diangkut seluruh armada, serta *objective function* lain yang tidak menambah variabel yang ada.]

1.5. Persoalan Uji

Empat persoalan uji dibangkitkan oleh core/generator.py dengan *seed* tetap, sehingga dapat dibuat ulang dengan `PYTHONPATH=src python -m core.generator`. Total volume paket diatur sekitar 1,2 kali volume truk (1,4 kali pada persoalan small) dan total berat 1,4 kali

kapasitas, sehingga tidak semua paket dapat diangkut. Sekitar 20% paket bersifat pecah-belah, ETA diacak antara 1 dan 7, dan *value* paket sebanding dengan volumenya dikali faktor acak 1 sampai 4 ditambah 5 sampai 25. Semua eksperimen wajib memakai persoalan medium.

Tabel 2 Persoalan uji

Berkas	Truk (w * l * h, kapasitas)	Paket	Fragile	Total berat	Volume paket / truk	Total value
small	1 truk: 6 * 8 * 4, 100	10	1	141	283 / 192	896
medium	1 truk: 10 * 20 * 5, 500	35	6	700	1.209 / 1.000	3.217
large	1 truk: 15 * 30 * 8, 1.500	56	10	2.100	4.339 / 3.600	11.444
multi_truck	3 truk: 8 * 12 * 4, 300; 6 * 10 * 4, 180; 10 * 16 * 5, 450	52	9	1.300	1.732 / 1.424	4.829

BAB II PEMBAHASAN

2.1. Representasi State dan Sistem Koordinat

Peti kemas truk ke-t menempati ruang [0, W) x [0, L) x [0, H) dengan sumbu x sebagai width, y sebagai length, dan z sebagai tinggi (vertikal). Posisi sebuah paket dinyatakan oleh sudut kiri-bawah-belakangnya (x, y, z). State adalah pemetaan seluruh paket ke penempatannya: indeks truk (atau kosong bila paket berada di luar), koordinat (x, y, z), dan orientasi. State bersifat immutable, sehingga setiap move menghasilkan salinan baru dan algoritma tidak dapat merusak state yang sedang dipegang algoritma lain.

Orientasi dinyatakan sebagai permutasi tiga huruf dari "xyz" yang menyatakan sumbu truk tempat sisi width, length, dan height paket sejajar. Orientasi "xyz" adalah bawaan, sedangkan "yxz" adalah hasil rotasi 90 derajat terhadap sumbu z. Rotasi terhadap sumbu z menukar x dan y, terhadap sumbu x menukar y dan z, dan terhadap sumbu y menukar x dan z. Rangkaian rotasi tersebut mencapai keenam orientasi yang mungkin.

Tabel 3 Contoh representasi penempatan paket

Paket	Truk	x	y	z	Orientasi	Dimensi Efektif
P01 (3, 4, 1)	0	0	0	0	xyz	(3, 4, 1)
P02 (2, 6, 3)	0	3	0	0	xyz	(6, 2, 3)
P03 (2, 2, 2)	kosong	-	-	-	xyz	(2, 2, 2)

Tabel 4 Fungsi dan kelas pada representasi

Nama fungsi/kelas	Deskripsi
Package, Truck, Problem	Model data untuk paket, truk, dan satu persoalan (daftar paket dan daftar truk)
Placement, State	Penempatan 1 paket (truks, x, y, z, orientasi, dimensi efektif) dan kumpulan penempatan seluruh paket
dims_for	Menghitung dimensi efekti (dx, dy, dz) paket untuk suatu orientasi
rotate	Rotasi 90 derajat terhadap sumbu x, y, atau z, yaitu

	menukar dua huruf sumbu pada orientasi
Load_problem, state_to_dict, state_from_dict	Membaca persoalan dari JSON serta serialisasi dan deserialisasi state ke format hasil run.

core/orientation.py

```
def dims_for(base_dims: tuple[int, int, int], orientation: str) ->
tuple[int, int, int]:
    return (
        base_dims[orientation.index("x")],
        base_dims[orientation.index("y")],
        base_dims[orientation.index("z")],
    )


def rotate(orientation: str, axis: str) -> str:
    if axis not in AXES:
        raise ValueError(f"sumbu harus salah satu dari 'xyz', bukan
{axis!r}")
    a, b = (c for c in AXES if c != axis)
    return orientation.translate(str.maketrans({a: b, b: a}))
```

core/models.py

```
class Placement(NamedTuple):
    truck: int | None
    x: int = 0
    y: int = 0
    z: int = 0
    orientation: str = "xyz"
    dims: tuple[int, int, int] = (0, 0, 0)

    @property
    def inside(self) -> bool:
        return self.truck is not None


@dataclass(frozen=True)
class State:
    ids: tuple[str, ...] = field(compare=False)
    placements: tuple[Placement, ...]

    def __len__(self) -> int:
        return len(self.placements)
```

2.2. Objective Function dan Penanganan Constraint

Nilai yang dimaksimalkan adalah skor paket di dalam truk dikurangi penalti pelanggaran:

f(s) = jumlah skor paket di dalam truk - λ * (overlap + out_of_bounds + floating + fragile_support) - jumlah untuk setiap truk yang kelebihan berat dari (ρ * kelebihan berat + λ)

Pada objective bawaan, skor sebuah paket adalah *value*-nya. Satuan penalti λ adalah skor terbesar di antara semua paket, sehingga mengeluarkan paket yang melanggar dari truk tidak pernah lebih buruk daripada mempertahankannya. Koefisien ρ adalah rasio skor terhadap berat terbesar, sehingga menambah paket ke truk yang sudah penuh tidak pernah menguntungkan.

Strategi penalti dipilih agar pencarian tetap dapat melewati *state* tidak valid dan memperoleh gradien menuju *state* valid (mengurangi satu pasangan *overlap*, misalnya, langsung menaikkan nilai *objective*). Pelaporan tidak tercampur dengan penalti. Artinya, total *value* selalu merupakan nilai murni paket di dalam truk, dan *feasible* hanya benar bila tidak ada pelanggaran sama sekali.

Tabel 5 Jenis pelanggaran constraint

Pelanggaran	Definisi
overlap	Jumlah pasangan paket di truk yang sama dengan volume yang saling beririsan
out_of_bounds	Jumlah paket yang tidak seluruhnya berada di dalam peti kemas
floating	Jumlah paket dengan z > 0 yang tidak memiliki satu pun titik alas tersangga (cukup satu titik alas tersangga agar tidak dihitung melayang)
fragile_support	Jumlah pasangan paket pecah-belah dan paket yang bertumpu langsung di atasnya
overweight	Total kelebihan berat di atas kapasitas, dijumlahkan seluruh truk

Pemeriksaan dilakukan per truk dengan operasi matriks (menggunakan *library* numpy) pada seluruh pasangan paket, sehingga satu evaluasi *objective* hanya memerlukan waktu yang sangat singkat.

core/constraint.py

```
def check_state(problem: Problem, state: State) -> Report:
    report = Report(len(problem.trucks))
    placements = state.placements
    for i, p in enumerate(placements):
        if p.truck is not None:
            report.inside[p.truck].append(i)

    weights, fragile = problem.weights, problem.fragile
    for t, members in enumerate(report.inside):
        if not members:
            continue
        truck = problem.trucks[t]
        load = sum(weights[i] for i in members)
        report.weight_per_truck[t] = load
        if load > truck.max_capacity:
            report.overweight += load - truck.max_capacity
            report.overweight_trucks += 1

        boxes = np.array([(p.x, p.y, p.z, *p.dims) for p in (placements[i]
for i in members)], dtype=np.int64)
        x, y, z, dx, dy, dz = boxes.T
        x2, y2, z2 = x + dx, y + dy, z + dz

        report.out_of_bounds += int(((x < 0) | (y < 0) | (z < 0)
                                     | (x2 > truck.w) | (y2 > truck.l) |
(z2 > truck.h)).sum())
        if len(members) == 1:
            if z[0] > 0:
                report.floating += 1
            continue

        xy = ((x[:, None] < x2[None, :]) & (x[None, :] < x2[:, None])
              & (y[:, None] < y2[None, :]) & (y[None, :] < y2[:, None]))
        np.fill_diagonal(xy, False)
        zo = (z[:, None] < z2[None, :]) & (z[None, :] < z2[:, None])
```

```
        report.overlap += int((xy & zo).sum()) // 2


        beneath = xy & (z2[None, :] == z[:, None])
        report.floating += int(((z > 0) & ~beneath.any(axis=1)).sum())
        frag = np.fromiter((fragile[i] for i in members), dtype=bool,
count=len(members))
        if frag.any():
            report.fragile_support += int(beneath[:, frag].sum())
    return report
```

core/objective.py

```
def _assess(problem: Problem, state: State, name: str):
    scoring = _scoring(problem, name)
    report = check_state(problem, state)
    value_per_truck = [0.0] * len(problem.trucks)
    volume_per_truck = [0] * len(problem.trucks)
    score = 0.0
    for t, members in enumerate(report.inside):
        for i in members:
            score += scoring.scores[i]
            value_per_truck[t] += problem.values[i]
            volume_per_truck[t] += problem.volumes[i]

    penalty = scoring.unit * (report.overlap + report.out_of_bounds
                              + report.floating + report.fragile_support)
    if report.overweight_trucks:
        penalty += scoring.rho * report.overweight + scoring.unit *
report.overweight_trucks
    objective = score - penalty

    if scoring.utilization_bonus:
        used = 0.0
        for t, truck in enumerate(problem.trucks):
            used += (min(1.0, volume_per_truck[t] / truck.volume)
                     + min(1.0, report.weight_per_truck[t] /
truck.max_capacity)) / 2
        objective += scoring.utilization_bonus * used / len(problem.trucks)
    return objective, report, value_per_truck
```

2.3 Neighbor Moves dan Initial State

Ada 3 jenis *move* yang diimplementasikan pada program ini, yaitu *swap*, *move*, dan rotate. *Swap* menukar "slot" dua paket (truk dan posisi), dengan syarat minimal salah satu berada di dalam truk. *Move* memindahkan sebuah paket ke posisi lain, ke truk lain, atau mengeluarkannya dari truk. *Rotate* memutar paket 90 derajat terhadap sumbu x, y, atau z, dan ditolak bila hasilnya menabrak dinding atau paket lain.

Ruang koordinat penuh terlalu besar untuk dienumerasi, sehingga posisi penempatan dibatasi ke titik yang bersentuhan dengan dinding atau paket lain (*extreme points*). Titik jangkar (x, y) diambil dari keempat sudut lantai serta sisi kanan, sisi depan, dan sudut paket yang sudah ada beserta proyeksinya ke dinding. Untuk setiap titik jangkar, paket "dijatuhkan" ke penyangga tertinggi di bawah alasnya, sehingga selalu tersangga dan tidak beririsan dengan paket lain. Titik dibuang jika paket akan melewati atap, jika penyangganya paket pecah-belah, atau jika paket yang dipindahkan pecah-belah dan ada paket yang bertumpu di atasnya. *Move* ke truk yang kapasitas beratnya tidak cukup juga tidak dibangkitkan.

Pada *swap*, kedua paket diangkat dulu, lalu masing-masing dijatuhkan pada salah satu dari empat perataan sudut di dalam *footprint* slot lawannya. *Swap* ditolak jika salah satu paket tidak memiliki perataan yang valid. Heuristik tambahan *gravity settle* (`settle`) menjatuhkan paket yang kehilangan penyangga setelah sebuah paket diangkat atau dirotasi, agar satu *move* tidak memunculkan banyak pelanggaran melayang.

Pada pencarian acak, jenis *move* dipilih dengan peluang swap 25%, move 55%, dan rotate 20%. Dengan peluang 10%, *move* berupa *move* bebas yang memilih koordinat seragam di seluruh peti kemas sehingga *state* tidak valid tetap dapat dijelajahi dan dihukum oleh *objective function* dengan peluang 15% paket yang berada di dalam dikeluarkan. Untuk algoritma yang memerlukan seluruh tetangga, tersedia *lazy enumeration* dengan urutan acak. Pada persoalan medium sebuah *state* memiliki rata-rata sekitar 277 tetangga, dan pada multi_truck sekitar 575 tetangga.

Fungsi `random_neighbor` memilih jenis *move* dengan peluang *swap* 25%, *move* 55%, dan *rotate* 20%. Dari *move*, 10% berupa *move* bebas yang memilih koordinat acak seragam di seluruh peti kemas. *Move* bebas sengaja dapat menghasilkan *state* tidak valid yang kemudian dihukum oleh penalti, sebagai sarana eksplorasi. Paket yang berada di dalam truk dikeluarkan dengan peluang 15%. Untuk algoritma yang memerlukan seluruh tetangga, `neighbors` membangkitkan tetangga secara malas (*lazy*) dengan urutan acak. Setiap paket dicoba dipindah ke setiap truk (maksimal empat titik kandidat acak per pasangan paket-truk), paket di dalam truk juga dicoba dikeluarkan dan dirotasi terhadap ketiga sumbu, dan setiap pasangan paket dicoba ditukar. Pada 40 *state* awal acak persoalan medium (*seed* 1 sampai 40), satu *state* memiliki rata-rata 265 tetangga (172 sampai 390), pada multi_truck rata-ratanya 601 tetangga (365 sampai 818).

Initial state dibangkitkan oleh `random_state`. Orientasi setiap paket diacak, urutan paket diacak, lalu setiap paket dengan peluang 0,5 ditempatkan pada titik kandidat acak di truk (urutan truk diacak) yang kapasitas beratnya masih cukup. Paket lain tetap di luar. Karena memakai titik kandidat, *initial state* yang dihasilkan selalu valid. Tersedia juga mode `uniform` yang memakai koordinat acak tanpa memeriksa *constraint*, tetapi mode ini tidak dipakai pada eksperimen.

Untuk memeriksa efek heuristik ini, kami membangkitkan 40 *state* awal acak pada persoalan medium, lalu memeriksa validitas setiap tetangga hasil `neighbors` dan 25 tetangga hasil `random_neighbor` dari masing-masing *state* (Tabel 6).

Tabel 6 Validasi *state* setelah satu langkah dari *state* valid (medium, 40 *state* awal acak)

Sumber	Jenis *move*	Sampel	Valid	Valid (%)
`neighbors`	Move ke titik kandidat	3.042	3.041	100,0
`neighbors`	Move keluar truk	616	615	99,8
`neighbors`	Rotate	505	498	98,6
`neighbors`	Swap	6.423	6.416	99,9
`random_neighbor`	Move ke titik kandidat	470	470	100,0
`random_neighbor`	Move keluar truk	50	49	98,0
`random_neighbor`	Move bebas (koordinat acak)	86	0	0,0
`random_neighbor`	Rotate	262	261	99,6
`random_neighbor`	Swap	132	132	100,0

Move kandidat, *swap*, dan *rotate* hampir selalu menghasilkan *state* valid, sehingga pencarian tidak membuang evaluasi pada *state* yang pasti buruk. *Move* bebas tidak pernah valid pada sampel ini dan hanya berfungsi sebagai sarana eksplorasi. Semua tetangga tidak valid hasil `neighbors` pada sampel ini (16 dari 10.586) melanggar *constraint* pecah-belah.

Penyebabnya adalah `settle`, yaitu kondisi saat sebuah paket diangkat atau dirotasi, paket yang bertumpu padanya dijatuhkan tanpa memeriksa apakah penyangga barunya pecah-belah.

Tabel 7 Fungsi dan kelas pada *neighbor moves* dan *initial state*

Nama fungsi/kelas	Deskripsi
`random_neighbor`	Satu tetangga acak beserta deskripsi move-nya
`neighbors`	Enumerasi tetangga secara malas dengan parameter batas limit
`TruckBoxes, candidate_positions`	Kumpulan balok di satu truk dan titik penempatan valid berbasis extreme points serta penjatuhan paket
`settle, has_dependents`	Menjatuhkan paket yang kehilangan penyangga, dan memeriksa apakah ada paket yang bertumpu pada suatu paket
`_swap, _land, _rotate, _moves_to, _free_move`	Implementasi swap, penempatan paket ke slot lawan, rotasi, move ke titik kandidat atau keluar truk, dan move bebas
`random_state`	Pembangkit initial state acak

core/candidates.py

```
def candidate_positions(boxes: TruckBoxes, truck, dims: tuple[int, int,
int], fragile: bool,
                        anchors: tuple[np.ndarray, np.ndarray] | None =
None) -> np.ndarray:
    dx, dy, dz = dims
    W, L, H = truck.w, truck.l, truck.h
    if dx > W or dy > L or dz > H:
        return _EMPTY
    if anchors is not None:
        ax, ay = np.asarray(anchors[0], dtype=np.int64),
np.asarray(anchors[1], dtype=np.int64)
    elif boxes.size == 0:
        return np.zeros((1, 3), dtype=np.int64)
    else:
        bx, by, bx2, by2 = boxes.x, boxes.y, boxes.x2, boxes.y2
        zero = np.zeros(boxes.size, dtype=np.int64)
        corners = np.array([0, W - dx, 0, W - dx], dtype=np.int64)
        ax = np.concatenate((corners, bx2, bx, bx, bx2, bx - dx, bx, zero))
        ay = np.concatenate((np.array([0, 0, L - dy, L - dy],
dtype=np.int64), by, by2, by, zero, by, by - dy, by2))

    inside = (ax >= 0) & (ay >= 0) & (ax <= W - dx) & (ay <= L - dy)
    ax, ay = ax[inside], ay[inside]
    if ax.size == 0:
        return _EMPTY
    _, first = np.unique(ax * (L + 1) + ay, return_index=True)
    ax, ay = ax[first], ay[first]
```

```
    if boxes.size == 0:
        return np.stack([ax, ay, np.zeros_like(ax)], axis=1)

    ov = ((ax[:, None] < boxes.x2[None, :]) & (boxes.x[None, :] < (ax +
dx)[:, None])
          & (ay[:, None] < boxes.y2[None, :]) & (boxes.y[None, :] < (ay +
dy)[:, None]))
    z = np.where(ov, boxes.z2[None, :], 0).max(axis=1)
    valid = z + dz <= H
    valid &= ~(ov & (boxes.z2[None, :] == z[:, None]) & boxes.fragile[None,
:]).any(axis=1)
    if fragile:
        valid &= ~(ov & (boxes.z[None, :] == (z + dz)[:,
None])).any(axis=1)
    return np.stack([ax, ay, z], axis=1)[valid]
```

core/candidates.py

```
def settle(placements: list[Placement], truck: int) -> None:
    order = sorted((i for i, p in enumerate(placements) if p.truck == truck
and p.z > 0),
                   key=lambda i: placements[i].z)
    if not order:
        return
    rest = [(p.x, p.y, p.x + p.dims[0], p.y + p.dims[1], p.z + p.dims[2])
            for p in placements if p.truck == truck and p.z <= 0]
    for i in order:
        p = placements[i]
        x2, y2 = p.x + p.dims[0], p.y + p.dims[1]
        base, supported = 0, False
        for ox, oy, ox2, oy2, top in rest:
            if p.x < ox2 and ox < x2 and p.y < oy2 and oy < y2:
                if top == p.z:
                    supported = True
                    break
                if top < p.z and top > base:
                    base = top
        z = p.z if supported else base
        if z != p.z:
            placements[i] = p._replace(z=z)
        rest.append((p.x, p.y, x2, y2, z + p.dims[2]))
```

core/moves.py

```
def _land(problem: Problem, items: list[Placement], i: int, slot:
Placement, rng: random.Random | None) -> bool:
    pkg, t = problem.packages[i], slot.truck
    dx, dy = items[i].dims[0], items[i].dims[1]
    xs = {slot.x, slot.x + slot.dims[0] - dx}
    ys = {slot.y, slot.y + slot.dims[1] - dy}
    ax = [x for x in sorted(xs) for _ in ys]
    ay = [y for _ in xs for y in sorted(ys)]
    boxes = TruckBoxes(problem, items, t)
    spots = candidate_positions(boxes, problem.trucks[t], items[i].dims,
pkg.is_fragile, anchors=(ax, ay))
    if len(spots) == 0:
        return False
    row = spots[rng.randrange(len(spots)) if rng is not None else 0]
```

```
    items[i] = problem.place(i, t, int(row[0]), int(row[1]), int(row[2]),
items[i].orientation)
    return True


def _swap(problem: Problem, state: State, a: int, b: int, rng:
random.Random | None) -> tuple[State, Move] | None:
    pa, pb = state.placements[a], state.placements[b]
    if pa.truck is None and pb.truck is None:
        return None
    items = list(state.placements)
    _lift(problem, items, a)
    _lift(problem, items, b)
    for i, slot in ((a, pb), (b, pa)):
        if slot.truck is not None and not _land(problem, items, i, slot,
rng):
            return None
    result = State(state.ids, tuple(items))
    if result == state:
        return None
    return result, {"type": "swap", "a": problem.ids[a], "b":
problem.ids[b]}
```

core/moves.py

```
def _rotate(problem: Problem, state: State, i: int, axis: str) ->
tuple[State, Move] | None:
    p = state.placements[i]
    orientation = rotate(p.orientation, axis)
    dims = dims_for(problem.packages[i].dims, orientation)
    if dims == p.dims:
        return None
    items = list(state.placements)
    x, y = p.x, p.y
    if p.truck is not None:
        truck = problem.trucks[p.truck]
        x, y = max(0, min(x, truck.w - dims[0])), max(0, min(y, truck.l -
dims[1]))
    items[i] = p._replace(x=x, y=y, orientation=orientation, dims=dims)
    if p.truck is not None:
        settle(items, p.truck)
        if not _clear(problem, items, i):
            return None
    move = {"type": "rotate", "package": problem.ids[i], "axis": axis,
"orientation": orientation}
    return State(state.ids, tuple(items)), move
```

core/moves.py

```
def random_neighbor(problem: Problem, state: State, rng: random.Random) ->
tuple[State, Move]:
    kinds, weights = list(KIND_WEIGHTS), list(KIND_WEIGHTS.values())
    for _ in range(MAX_TRIES):
        result = _RANDOM[rng.choices(kinds, weights)[0]](problem, state,
rng)
        if result is not None:
            return result
```

```
    if len(state):
        return _free_move(problem, state, rng.randrange(len(state)), rng)
    return state, {"type": "none"}
```

core/initializer.py

```
def random_state(problem: Problem, rng: random.Random, mode: str =
"feasible",
                 p_inside: float = DEFAULT_P_INSIDE) -> State:
    if mode not in ("feasible", "uniform"):
        raise ValueError(f"mode harus 'feasible' atau 'uniform', bukan
{mode!r}")
    placements = [problem.outside(i, rng.choice(ORIENTATIONS)) for i in
range(len(problem.packages))]
    order = list(range(len(placements)))
    rng.shuffle(order)
    trucks = range(len(problem.trucks))

    for i in order:
        if rng.random() >= p_inside:
            continue
        pkg, dims, orientation = problem.packages[i], placements[i].dims,
placements[i].orientation
        if mode == "uniform":
            t = rng.choice(trucks)
            truck = problem.trucks[t]
            x, y, z = (rng.randint(0, max(0, lim - d)) for lim, d in
zip(truck.dims, dims))
            placements[i] = problem.place(i, t, x, y, z, orientation)
            continue
        for t in rng.sample(trucks, len(trucks)):
            truck = problem.trucks[t]
            load = sum(problem.weights[j] for j, p in enumerate(placements)
if p.truck == t)
            if load + pkg.weight > truck.max_capacity:
                continue
            cands = candidate_positions(TruckBoxes(problem, placements, t),
truck, dims, pkg.is_fragile)
            if len(cands):
                x, y, z = (int(v) for v in
cands[rng.randrange(len(cands))])
                placements[i] = problem.place(i, t, x, y, z, orientation)
                break
    return State(problem.ids, tuple(placements))
```

2.4 Kerangka Algoritma

Semua algoritma pencarian, yaitu lima varian Hill-climbing, Simulated Annealing, dan Genetic Algorithm, mewarisi kelas dasar LocalSearch pada algorithms/base.py. Kelas ini mengerjakan bagian yang sama untuk setiap algoritma, yakni memeriksa parameter, membuat pembangkit bilangan acak dari seed, mengambil fungsi dari core, mengukur durasi, dan

menyusun hasil run. Setiap subkelas cukup mengisi method `_search()` yang mengembalikan pasangan state awal dan state akhir.

Saat objek algoritma dibuat, LocalSearch menolak parameter yang tidak dikenal, menggabungkan parameter masukan dengan `default_params`, membuat `random.Random(seed)`, dan mengambil fungsi objective sesuai namanya (default atau alt). Method `run()` membuat ulang pembangkit acak dari seed yang sama dan mengosongkan log, lalu menjalankan `_search()` sambil mengukur waktu dengan `time.perf_counter()`.

Karena pembangkit acak selalu dibuat ulang, satu seed selalu menghasilkan run yang sama. Setelah pencarian selesai, _build_result() mengevaluasi state awal dan akhir dengan core.evaluate, mengubah keduanya menjadi dict, lalu menambahkan banyak pemanggilan fungsi objective (evaluations) dan value per truk ke extra. Evaluasi akhir ini tidak ikut dihitung dalam durasi.

Algoritma hanya berhubungan dengan core lewat empat method, yaitu `objective(state)`, `random_state()`, `random_neighbor(state)`, dan `neighbors(state, limit)`. Method `objective()` sekaligus menambah penghitung evaluasi, sehingga biaya komputasi semua algoritma dapat dibandingkan dengan satuan yang sama. Fungsi core diambil oleh `load_core()` berdasarkan daftar CORE_API sehingga pengujian dapat memakai core tiruan tanpa mengubah kode algoritma.

Setiap run menghasilkan satu RunResult yang disimpan sebagai satu berkas JSON di folder output (Tabel 8). Plot, program visualisasi, dan rekap membaca berkas yang sama, jadi algoritma tidak perlu dijalankan ulang untuk membuat grafik atau tabel. Elemen pertama history["objective"] berisi objective state awal, diikuti satu nilai untuk setiap iterasi, sehingga panjangnya selalu iterations + 1. Simulated Annealing menambahkan seri acceptance_prob dan temperature, sedangkan Genetic Algorithm menambahkan max dan avg.

Tabel 8 Isi hasil run (RunResult)

Kelompok	Isi
Identitas	algorithm, label, objective_name, problem (salinan persoalan), problem_file, seed, params
State awal	initial_state, initial_objective, initial_value, initial_feasible, initial_violations

State akhir	final_state, final_objective, final_value, feasible, final_violations
Proses	duration_sec, iterations, history (riwayat objective dan seri lain), extra (data khusus algoritma, termasuk evaluations dan value_per_truck)

Algoritma didaftarkan pada registry.py dengan nama kanonik dan alias, misalnya steepest_ascent_hc dengan alias sahc dan simulated_annealing dengan alias sa. Pendaftaran ini dipakai oleh dua titik masuk program. main.py menjalankan satu algoritma pada satu persoalan, contohnya python src/main.py --problem src/data/medium.json --algo sa --seed 1 --out output/manual. run_experiments.py membaca berkas konfigurasi berisi persoalan, objective, daftar seed, dan daftar run; contohnya adalah experiments/configs/hill_climbing.json di akhir subbab ini. Parameter pada grid diekspansi menjadi semua kombinasinya, dan setiap kombinasi dijalankan untuk setiap seed. Hasil disimpan sebagai <label>_<seed>.json, dan konfigurasi yang membuat dua run menulis ke berkas yang sama ditolak. Run yang gagal tidak menghentikan run lain; daftarnya dicetak di akhir. Terakhir, summarize.py memindai seluruh hasil di output dan menulis summary.csv (satu baris per run), summary_by_config.csv (rata-rata dan simpangan baku per konfigurasi), dan summary.md.

Tabel 9 Fungsi dan kelas pada kerangka algoritma

Nama fungsi/kelas	Deskripsi
RunResult	Dataclass berisi seluruh field hasil run, dengan to_dict, from_dict, to_json, save, dan load. Array angka ditulis dalam satu baris agar berkas JSON ringkas.
LocalSearch	Kelas dasar semua algoritma. Method run() adalah template method; subkelas hanya mengimplementasikan _search(). Method record(series, value) menambah nilai ke riwayat.
load_core, CORE_API	Mengambil fungsi API core (load_problem, state_to_dict, random_state, evaluate, get_objective, random_neighbor, neighbors) dari modulnya
registry.py	ALGORITHMS, canonical_name (menerima nama atau alias, tidak peka huruf besar), get_algorithm, dan create

main.py	Antarmuka baris perintah untuk satu run dengan argumen --list, --problem, --algo, --objective, --seed, --params, --param, --label, dan --out
run_experiments.py	expand_config mengubah konfigurasi menjadi daftar run dan execute menjalankan satu run; opsi --dry-run, --skip-existing, dan --only mengatur eksekusi
summarize.py	load_runs, run_row, group_rows, dan summarize menyusun rekap CSV dan Markdown

algorithms/base.py (kelas LocalSearch)

```
class LocalSearch(ABC):
      name: ClassVar[str] = "local_search"
      aliases: ClassVar[tuple[str, ...]] = ()
      default_params: ClassVar[dict[str, Any]] = {}

      def __init__(self, problem, objective: str = "default", seed: int |
None = None, **params):
      unknown = sorted(set(params) - set(self.default_params))
      if unknown:
            raise ValueError(
            f"{self.name}: parameter tidak dikenal {unknown}; "
            f"parameter yang tersedia: {sorted(self.default_params)}"
            )
      self.core = load_core()
      self.problem = problem
      self.objective_name = objective
      self.seed = seed
      self.params = {**self.default_params, **params}
      self.rng = random.Random(seed)
      self._objective_fn = self.core.get_objective(objective)
      self._reset_log()

      def _reset_log(self) -> None:
      self.iterations = 0
      self.evaluations = 0
      self.history: dict[str, list] = {"objective": []}
      self.extra: dict[str, Any] = {}

      def objective(self, state) -> float:
      self.evaluations += 1
      return self._objective_fn(self.problem, state)

      def random_state(self):
      return self.core.random_state(self.problem, self.rng)

      def random_neighbor(self, state):
      return self.core.random_neighbor(self.problem, state, self.rng)

      def neighbors(self, state, limit: int | None = None) -> Iterable:
      return self.core.neighbors(self.problem, state, rng=self.rng,
limit=limit)
```

```
      def record(self, series: str, value) -> None:
      self.history.setdefault(series, []).append(value)

      def run(self) -> RunResult:
      self.rng = random.Random(self.seed)
      self._reset_log()
      start = time.perf_counter()
      initial, final = self._search()
      duration = time.perf_counter() - start
      return self._build_result(initial, final, duration)

      @abstractmethod
      def _search(self) -> tuple[Any, Any]:
      ...

      def _build_result(self, initial, final, duration: float) ->
RunResult:
      ev0 = self.core.evaluate(self.problem, initial,
objective=self.objective_name)
      ev1 = self.core.evaluate(self.problem, final,
objective=self.objective_name)
      extra = {**self.extra, "evaluations": self.evaluations}
      value_per_truck = getattr(ev1, "value_per_truck", None)
      if value_per_truck is not None:
            extra["value_per_truck"] = list(value_per_truck)
      return RunResult(
            algorithm=self.name,
            objective_name=self.objective_name,
            seed=self.seed,
            params=dict(self.params),
            initial_state=self.core.state_to_dict(initial),
            initial_objective=float(ev0.objective),
            initial_value=float(ev0.total_value),
            initial_feasible=bool(ev0.feasible),
            initial_violations=dict(ev0.violations),
            final_state=self.core.state_to_dict(final),
            final_objective=float(ev1.objective),
            final_value=float(ev1.total_value),
            feasible=bool(ev1.feasible),
            final_violations=dict(ev1.violations),
            duration_sec=duration,
            iterations=self.iterations,
            history=self.history,
            extra=extra,
      )
```

2.5 Hill-climbing

Bonus 2 dikerjakan dengan mengimplementasikan seluruh varian Hill-climbing yang dibahas pada materi kuliah dan spesifikasi tugas, yaitu Steepest Ascent, Stochastic, Sideways Move, dan Random Restart. Pada semua varian, tetangga dibangkitkan oleh core.neighbors (subbab 2.3) dan satu iterasi berarti satu langkah pindah ke state lain.

Tiga varian (Steepest Ascent, Stochastic, dan Sideways Move) mewarisi kelas HillClimbing pada hill_climbing/common.py. Method _search() di kelas ini membangkitkan satu state acak lalu memanggil climb(state). Pendakian sengaja dipisah ke climb agar Random Restart dapat memakai varian mana pun sebagai pendaki. Kelas ini juga menyediakan beberapa helper: _start mencatat objective awal, _advance menambah iterasi dan mencatat objective, _below_limit memeriksa max_iterations (None berarti tanpa batas), dan best_neighbor mengevaluasi seluruh tetangga lalu mengembalikan yang terbaik. Jika beberapa tetangga sama-sama terbaik, salah satunya dipilih acak supaya pencarian tidak bolak-balik di antara dua state pada dataran (plateau). Semua perbandingan nilai memakai toleransi EPS = 1e-9 untuk menghindari salah deteksi akibat pembulatan bilangan pecahan.

Tabel 10 Varian Hill-climbing dan parameter yang dipakai

Varian	**Cara memilih tetangga**	**Parameter eksperimen**	**Kriteria berhenti**
Steepest Ascent	Tetangga terbaik dari seluruh *neighborhood*	`neighbor_limit = null`, `max_iterations = null`	Tidak ada tetangga yang lebih baik
Stochastic	Acak seragam di antara semua tetangga yang lebih baik (*uphill*)	`mode = uphill`, `weighted = false`, `max_iterations = 1000`	Tidak ada tetangga yang lebih baik, atau 1000 iterasi
Sideways Move	Tetangga terbaik; tetangga bernilai sama tetap diterima	`max_sideways = 100`	Tetangga terbaik lebih buruk, atau total *sideways move* mencapai 100

Steepest Ascent Hill-climbing

Pada setiap iterasi, algoritma memilih tetangga terbaik. Jika nilainya tidak lebih besar dari state sekarang, pencarian berhenti dengan stop_reason = "local_optimum". Banyak iterasi yang diminta spesifikasi dicatat pada iterations.

1. Bangkitkan state acak s.
2. Evaluasi seluruh tetangga s dan ambil yang terbaik, s'. Jika ada beberapa yang sama-sama terbaik, pilih satu secara acak.
3. Jika f(s') tidak lebih besar dari f(s), berhenti dan kembalikan s.
4. Ganti s dengan s', lalu ulangi dari langkah 2.

algorithms/hill_climbing/common.py (best_neighbor) dan steepest_ascent.py (climb)

```
def best_neighbor(self, state):
    best, best_value = [], -math.inf
    for neighbor, _ in self.neighbors(state,
limit=self.params.get("neighbor_limit")):
```

```
        value = self.objective(neighbor)
        if value > best_value + EPS:
            best, best_value = [neighbor], value
        elif value >= best_value - EPS:
            best.append(neighbor)
    if not best:
        return None, -math.inf
    return self.rng.choice(best), best_value

def climb(self, current):
    current_value = self._start(current)
    steps = 0
    while self._below_limit(steps):
        neighbor, neighbor_value = self.best_neighbor(current)
        if neighbor is None or neighbor_value <= current_value + EPS:
            self.extra["stop_reason"] = "local_optimum"
            return current
        current, current_value = neighbor, neighbor_value
        steps += 1
        self._advance(current_value)
    self.extra["stop_reason"] = "max_iterations"
    return current
```

Stochastic Hill-climbing

Mode yang dipakai pada eksperimen adalah uphill. Algoritma mengevaluasi seluruh tetangga, mengumpulkan yang lebih baik dari state sekarang, lalu memilih satu secara acak seragam. Dengan weighted = true, peluang pemilihan sebanding dengan besar kenaikan. Pencarian berhenti jika tidak ada tetangga yang lebih baik atau max_iterations tercapai. Tersedia juga mode random yang membangkitkan satu tetangga acak per iterasi dan berpindah hanya jika tetangga itu lebih baik, tetapi mode ini tidak dipakai pada eksperimen.

1. Bangkitkan state acak s.
2. Kumpulkan semua tetangga yang nilainya lebih besar dari f(s).
3. Jika kumpulan itu kosong, berhenti dan kembalikan s.
4. Ganti s dengan satu tetangga yang dipilih acak dari kumpulan tersebut.
5. Ulangi dari langkah 2 sampai batas iterasi.

algorithms/hill_climbing/stochastic.py (_climb_uphill)

```
def _climb_uphill(self, current):
    current_value = self._start(current)
    steps = 0
    while self._below_limit(steps):
        uphill = []
        for neighbor, _ in self.neighbors(current,
```

```
limit=self.params["neighbor_limit"]):
            value = self.objective(neighbor)
            if value > current_value + EPS:
            uphill.append((neighbor, value))
        if not uphill:
            self.extra["stop_reason"] = "local_optimum"
            return current
        if self.params["weighted"]:
            weights = [value - current_value for _, value in uphill]
            current, current_value = self.rng.choices(uphill,
weights=weights)[0]
        else:
            current, current_value = self.rng.choice(uphill)
        steps += 1
        self._advance(current_value)
    self.extra["stop_reason"] = "max_iterations"
    return current
```

Hill-climbing with Sideways Move

Varian ini sama dengan Steepest Ascent, tetapi tetangga terbaik yang bernilai sama dengan state sekarang tetap diterima sebagai sideways move. Sesuai kalimat spesifikasi, batas max_sideways dihitung secara total, bukan berturut-turut, dan pencarian langsung berhenti ketika jumlah sideways move mencapai batas tersebut (stop_reason = "max_sideways"). Pencarian juga berhenti jika tetangga terbaik lebih buruk. Banyak sideways move yang terpakai dicatat pada extra.sideways_used.

1. Bangkitkan state acak s dan atur penghitung sideways menjadi nol.
2. Pilih tetangga terbaik s'. Jika f(s') lebih kecil dari f(s), berhenti.
3. Jika f(s') sama dengan f(s) dan penghitung sudah mencapai max_sideways, berhenti.
4. Ganti s dengan s'. Jika langkah tadi adalah sideways move, tambah penghitung, dan berhenti bila penghitung mencapai max_sideways.
5. Ulangi langkah 2.

algorithms/hill_climbing/sideways.py (climb)

```
def climb(self, current):
      max_sideways = self.params["max_sideways"]
      current_value = self._start(current)
      steps = sideways = 0
      reason = "max_iterations"
      while self._below_limit(steps):
      neighbor, neighbor_value = self.best_neighbor(current)
      if neighbor is None or neighbor_value < current_value - EPS:
            reason = "local_optimum"
            break
      is_sideways = neighbor_value <= current_value + EPS
```

```
      if is_sideways and sideways >= max_sideways:
            reason = "max_sideways"
            break
      current, current_value = neighbor, neighbor_value
      steps += 1
      self._advance(current_value)
      if is_sideways:
            sideways += 1
            if sideways >= max_sideways:
            reason = "max_sideways"
            break
      self.extra["max_sideways"] = max_sideways
      self.extra["sideways_used"] = self.extra.get("sideways_used", 0) +
sideways
      self.extra["stop_reason"] = reason
      return current
```

Random Restart Hill-climbing

Random Restart menjalankan pendakian dari state awal acak yang berbeda sebanyak 1 + max_restart kali, karena pendakian pertama tidak dihitung sebagai restart. Pendaki bawaannya Steepest Ascent dan dapat diganti lewat parameter base_algorithm. Pendaki memakai pembangkit acak yang sama dengan Random Restart, jadi satu seed menentukan seluruh rangkaian restart. State akhir adalah state terbaik dari semua pendakian, sedangkan state awal adalah state awal pendakian pertama. Riwayat objective adalah gabungan riwayat semua pendakian; titik awal pendakian kedua dan seterusnya tidak dicatat, sehingga batas antar-restart berada pada jumlah kumulatif iterations_per_restart. Metrik yang diminta spesifikasi, yaitu banyak restart dan banyak iterasi per restart, dicatat pada extra.restarts dan extra.iterations_per_restart. Nilai iterations adalah total langkah dari semua pendakian.

1. Kosongkan state terbaik global.
2. Ulangi sebanyak max_restart + 1 kali: bangkitkan state acak baru, jalankan pendaki sampai berhenti, catat banyak iterasi dan nilai akhirnya, lalu simpan hasilnya jika lebih baik dari state terbaik global.
3. Kembalikan state terbaik global.

algorithms/hill_climbing/random_restart.py (_search)

```
def _search(self):
      climber = self._new_climber()
      climber.rng = self.rng
      initial = best = None
      best_value = -math.inf
      iterations_per_restart, start_objectives, final_objectives = [], [],
[]
```

```
      for attempt in range(self.params["max_restart"] + 1):
      climber.history = {"objective": []}
      start = self.random_state()
      before = climber.iterations
      final = climber.climb(start)
      values = climber.history["objective"]
        self.history["objective"].extend(values if attempt == 0 else
values[1:])
        iterations_per_restart.append(climber.iterations - before)
      start_objectives.append(values[0])
      final_objectives.append(values[-1])
      if initial is None:
            initial = start
      if values[-1] > best_value:
            best, best_value = final, values[-1]

      self.iterations = climber.iterations
      self.evaluations = climber.evaluations
      self.extra.update(
        max_restart=self.params["max_restart"],
      restarts=len(iterations_per_restart) - 1,
        iterations_per_restart=iterations_per_restart,
        start_objective_per_restart=start_objectives,
      objective_per_restart=final_objectives,
        best_restart=final_objectives.index(best_value),
      base_extra=dict(climber.extra),
      stop_reason="max_restart",
      )
      return initial, best
```

2.6 Simulated Annealing

Implementasi mengikuti pseudocode Simulated Annealing pada materi kuliah dengan jadwal pendinginan geometrik, $T_k = t_0 \times \text{cooling}^k$. Pada setiap iterasi dibangkitkan satu tetangga acak dengan random_neighbor, lalu dihitung $\Delta E = f(\text{tetangga}) - f(\text{sekarang})$. Tetangga dengan $\Delta E \geq 0$ selalu diterima, sedangkan tetangga yang lebih buruk diterima dengan peluang $e^{(\Delta E/T)}$. Pencarian berhenti ketika $T \leq$ t_min atau max_iterations tercapai. Pada Simulated Annealing, satu iterasi berarti satu tetangga yang dibangkitkan, baik diterima maupun tidak.

Setiap iterasi mencatat tiga seri: objective (nilai state sekarang setelah iterasi), acceptance_prob, dan temperature. Nilai acceptance_prob adalah $\min(1, e^{(\Delta E/T)})$, jadi bernilai 1 untuk $\Delta E \geq 0$. Seri ini dipakai untuk plot $e^{(\Delta E/T)}$ terhadap iterasi.

Frekuensi stuck di local optima dihitung melalui satu kejadian stuck dicatat setiap kali stuck_patience iterasi berturut-turut berlalu tanpa satu pun move naik ($\Delta E > 0$) yang diterima. Penghitung streak di-reset setelah kejadian dicatat dan setiap kali ada move naik yang diterima. Jumlah kejadian disimpan pada extra.stuck_count, iterasi saat kejadian tercatat pada extra.stuck_iterations, dan definisinya pada extra.stuck_definition. State akhir adalah state terakhir yang dicapai (return_best = false), sesuai materi kuliah. Objective terbaik yang pernah dikunjungi tetap dicatat pada extra.best_objective_seen.

Dengan parameter pada Tabel 11, T turun dari 50 ke 0,01 dalam 42.582 iterasi, yaitu ln(t_min/t0)/ln(cooling), sehingga batas max_iterations tidak pernah tercapai. Parameter ini dipilih setelah uji coba: dengan $t_0 = 100$ dan cooling = 0,9995 (sekitar 18 ribu iterasi), value akhir seed 1 adalah 2.479, sedangkan dengan $t_0 = 50$ dan cooling = 0,9998 value-nya menjadi 2.584.

Tabel 11 Parameter Simulated Annealing

Parameter	Nilai	Keterangan
t_0	50	Temperatur awal
cooling	0,9998	Faktor pendinginan geometrik
t_{min}	0,01	Temperatur minimum (kriteria berhenti)
max_iterations	100.000	Batas iterasi (tidak tercapai karena t_min tercapai lebih dulu)
stuck_patience	500	Panjang streak tanpa move naik untuk satu kejadian stuck
return_best	false	State akhir adalah state terakhir

1. Bangkitkan state acak s dan atur T = t0.
2. Selama $T > t_{min}$ dan iterasi < max_iterations: bangkitkan tetangga acak s' dan hitung $\Delta E = f(s') - f(s)$.
3. Jika $\Delta E \geq 0$, terima s'. Jika $\Delta E < 0$, terima s' dengan peluang e^(ΔE/T).
4. Catat objective, e^(ΔE/T), dan T, lalu perbarui penghitung stuck.
5. Ubah T menjadi T × cooling dan ulangi dari langkah 2
6. Kembalikan state terakhir.

2.7 Genetic Algorithm

Kromosom adalah *state* itu sendiri sehingga konsisten dengan representasi wajib. Populasi awal dibangkitkan dengan *initial state* acak dan *fitness* adalah nilai *objective function*. Seleksi orang tua memakai *tournament* berukuran 3. *Crossover* dilakukan seragam per paket (tiap paket mewarisi penempatan dari salah satu orang tua) dengan peluang 0,8. Mutasi memakai satu *move* acak dari *core* (swap, move, atau rotate) dengan peluang 0,2. Elitisme mempertahankan dua individu terbaik pada setiap generasi. *Crossover* dapat menghasilkan anak yang tidak valid; hal ini ditangani oleh penalti pada *objective function* sehingga individu tersebut cenderung tersingkir pada seleksi.

Dua parameter utama yang divariasikan pada eksperimen adalah jumlah populasi dan banyak iterasi (generasi). *State* awal yang dicatat adalah individu terbaik populasi awal, dan riwayat yang dicatat adalah nilai maksimum dan rata-rata populasi pada setiap generasi.

Tabel 12 Parameter Genetic Algorithm

Parameter	Nilai	Keterangan
population_size	15, 30, 60	Jumlah populasi (divariasikan)
generations	20, 40, 60	Banyak iterasi/generasi (divariasikan)
crossover_rate	0,8	Peluang *crossover* seragam per paket
mutation_rate	0,2	Peluang mutasi tiap anak
tournament_size	3	Ukuran *tournament* seleksi
elitism	2	Jumlah individu terbaik yang dipertahankan

Pseudocode Genetic Algorithm

1. Bangkitkan populasi awal dari N *state* acak dan hitung *fitness*-nya.
2. Untuk setiap generasi: salin E individu terbaik ke populasi baru (elitisme).
3. Selama populasi baru belum berisi N individu: pilih dua orang tua dengan *tournament*, lakukan *crossover* seragam per paket dengan peluang 0,8, lalu mutasi setiap anak dengan peluang 0,2.
4. Hitung *fitness* populasi baru dan catat nilai maksimum dan rata-ratanya.
5. Setelah seluruh generasi selesai, kembalikan individu terbaik sepanjang pencarian.

Kode 2.18 Seleksi, crossover, dan mutasi (algorithms/genetic_algorithm.py)

```
def _tournament_select(self, population: list[State], fitness:
list[float]) -> State:
        k = min(int(self.params["tournament_size"]), len(population))
        contenders = self.rng.sample(range(len(population)), k)
```

```
        winner = max(contenders, key=lambda idx: fitness[idx])
        return population[winner]

def _crossover(self, parent1, parent2):
        if self.rng.random() > float(self.params["crossover_rate"]):
            return parent1, parent2

        a_items = list(parent1.placements) if hasattr(parent1,
"placements") else list(parent1)
        b_items = list(parent2.placements) if hasattr(parent2,
"placements") else list(parent2)
        items1, items2 = [], []
        for a, b in zip(a_items, b_items):
            if self.rng.random() < 0.5:
                items1.append(a)
                items2.append(b)
            else:
                items1.append(b)
                items2.append(a)
        if not hasattr(parent1, "placements"):
            return tuple(items1), tuple(items2)
        return (State(self.problem.ids, tuple(items1)),
                State(self.problem.ids, tuple(items2)))

def _mutate(self, individual):
        if self.rng.random() < float(self.params["mutation_rate"]):
            mutated, _ = self.random_neighbor(individual)
            return mutated
          return individual
```

Kode 2.19 Alur utama Genetic Algorithm (algorithms/genetic_algorithm.py)

```
def _search(self) -> tuple[State, State]:
        pop_size = int(self.params["population_size"])
        generations = int(self.params["generations"])
        elitism_count = max(0, min(int(self.params["elitism"]), pop_size))

        population: list[State] = [self.random_state() for _ in
range(pop_size)]
        fitness: list[float] = [self.objective(ind) for ind in population]

        best_idx = max(range(pop_size), key=lambda i: fitness[i])
        initial_best = population[best_idx]
        best_overall: State = initial_best
        best_overall_score: float = fitness[best_idx]

        self.iterations = 0
        self.record("objective", float(best_overall_score))
        self.record("max", float(max(fitness)))
        self.record("avg", float(sum(fitness) / pop_size))

        for gen in range(1, generations + 1):
            ranked = sorted(range(pop_size), key=lambda i: fitness[i],
reverse=True)
            new_population: list[State] = [population[i] for i in
ranked[:elitism_count]]

            while len(new_population) < pop_size:
```

```
                c1, c2 = self._crossover(
                    self._tournament_select(population, fitness),
                    self._tournament_select(population, fitness),
                )
                new_population.append(self._mutate(c1))
                if len(new_population) < pop_size:
                    new_population.append(self._mutate(c2))

            population = new_population
            fitness = [self.objective(ind) for ind in population]

            gen_best_idx = max(range(pop_size), key=lambda i: fitness[i])
            if fitness[gen_best_idx] > best_overall_score:
                best_overall = population[gen_best_idx]
                best_overall_score = fitness[gen_best_idx]

            self.iterations = gen
            self.record("objective", float(best_overall_score))
            self.record("max", float(max(fitness)))
            self.record("avg", float(sum(fitness) / pop_size))

        self.extra["population_size"] = pop_size
        self.extra["generations"] = generations

        return initial_best, best_overall
```

2.8 Bonus 1: Objective Function Alternatif

Objective function alternatif (`alt`) hanya memakai atribut yang sudah ada, yaitu *value*, *weight*, dimensi, dan ETA, tanpa menambah variabel. Skor setiap paket adalah *value* × (1 + urgensi), dengan urgensi = (ETA maksimum − ETA paket) / (ETA maksimum − ETA minimum). Dengan begitu, paket paling mendesak (ETA terkecil) dihargai dua kali *value*-nya dan paket dengan ETA terbesar dihargai sebesar *value*-nya. Ditambahkan pula bonus utilisasi sebesar 5% dari total skor seluruh paket dikali rata-rata utilisasi truk, yaitu rata-rata utilisasi volume dan utilisasi berat setiap truk. Penalti pelanggaran memakai skema yang sama dengan *objective* bawaan, dengan λ dan ρ dihitung dari skor `alt`.

Objective ini dapat dipasang pada semua algoritma tanpa mengubah kode algoritma, cukup dengan argumen `--objective alt` atau field `objective` pada berkas konfigurasi. Karena skalanya berbeda, nilai *objective* `alt` tidak dapat dibandingkan langsung dengan *objective* bawaan; perbandingan pada subbab 2.14 memakai total *value* murni.

core/objective.py

```
def _scoring(problem: Problem, name: str) -> _Scoring:
    key = ("scoring", name)
    cached = problem.cache.get(key)
```

```
    if cached is not None:
        return cached
    if name == "default":
        scores = tuple(float(v) for v in problem.values)
        bonus = 0.0
    elif name == "alt":
        etas = [p.eta for p in problem.packages]
        lo, hi = min(etas, default=0), max(etas, default=0)
        span = hi - lo
        scores = tuple(p.value * (1 + ((hi - p.eta) / span if span else
0.0)) for p in problem.packages)
        bonus = ALT_UTILIZATION_WEIGHT * sum(scores)
    else:
        raise ValueError(f"objective tidak dikenal: {name!r}; pilihan:
{OBJECTIVES}")
    unit = max(scores, default=1.0) or 1.0
    rho = max((s / w for s, w in zip(scores, problem.weights) if w > 0),
default=1.0)
    result = problem.cache[key] = _Scoring(scores, unit, rho, bonus)
    return result
```

2.9 Bonus 3: Multi-truk

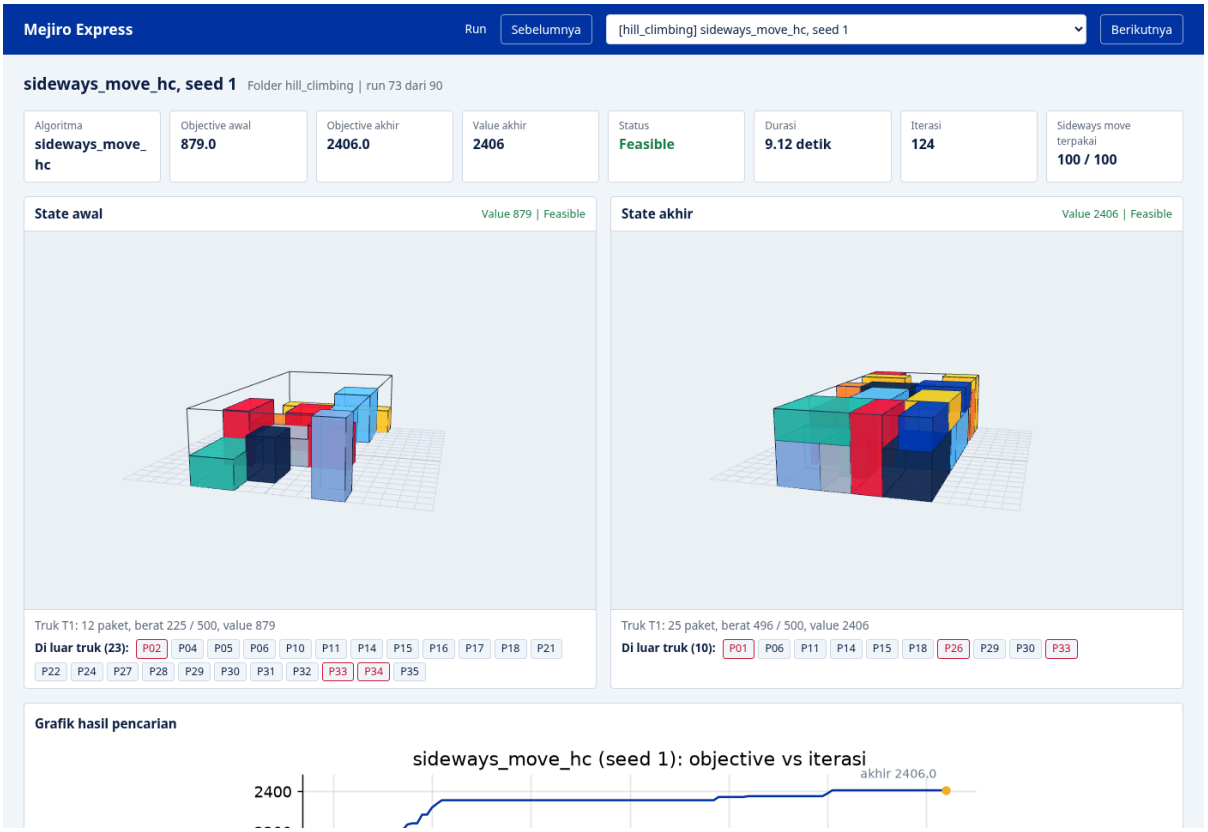
Persoalan dapat memuat lebih dari satu truk dengan ukuran dan kapasitas berbeda. Representasi dirancang sejak awal sebagai daftar truk, sehingga persoalan satu truk adalah kasus khusus dengan satu elemen. Setiap penempatan menyimpan indeks truknya, dan `check_state` memeriksa *overlap*, dinding, penyangga, paket pecah-belah, serta kapasitas secara terpisah untuk setiap truk. *Move* dapat memindahkan paket ke titik kandidat di truk mana pun, dan *swap* dapat menukar paket antartruk. *Initial state* acak mencoba truk dalam urutan acak untuk setiap paket.

Objective function adalah total *value* seluruh armada, dan *value* serta berat setiap truk dilaporkan pada `value_per_truck` dan `weight_per_truck`. Semua algoritma dapat dijalankan pada persoalan multi-truk tanpa perubahan kode. Program visualisasi menyediakan pilihan truk agar setiap peti kemas dapat dilihat satu per satu.

2.10 Program Visualisasi

Program visualisasi berupa aplikasi web yang berjalan lokal (python src/visualization/app.py output) dan tidak memerlukan koneksi internet. Aplikasi menampilkan *state* awal dan *state* akhir secara berdampingan dalam bentuk tiga dimensi. Paket di dalam peti kemas digambar sebagai balok (paket pecah-belah berwarna merah) dan keterangan paket muncul saat kursor diarahkan. Paket yang berada di luar peti kemas

ditampilkan sebagai daftar. Nilai *state*, berat terhadap kapasitas, dan status *feasible* ditampilkan pada setiap *state*. Untuk persoalan multi-truk tersedia pemilihan truk. Aplikasi juga menampilkan metrik *run* (termasuk metrik khusus algoritma) dan grafik hasil pencarian, serta tombol untuk menelusuri seluruh *run* yang ada.



Gambar 1 Tampilan program visualisasi (Sideways Move Hill-climbing, seed 1)

2.11 Hasil Eksperimen Hill-climbing

Setiap varian dijalankan tiga kali dengan seed 1, 2, dan 3 pada persoalan medium memakai objective bawaan. Untuk seed yang sama, semua varian Hill-climbing dan Simulated Annealing mulai dari state awal yang sama, dengan objective awal 879 (seed 1), 1.107 (seed 2), dan 1.039 (seed 3). State awal ini sudah feasible, jadi perbedaan hasil berasal dari cara algoritma mencari. Kedua belas run berakhir feasible.

Tabel 13 Ringkasan hasil Hill-climbing (rata-rata tiga run)

Varian	Value rata-rata	Std	Min	Maks	Iterasi	Evaluasi	Durasi (s)
Steepest Ascent	2.262,33	18,15	2.243	2.279	16,0	3.688	1,21
Stochastic	2.192,67	81,13	2.126	2.283	27,0	6.180	1,99

Sideways Move	2.395,00	16,52	2.376	2.406	121,0	13.347	8,80
Random Restart	2.365,67	85,91	2.297	2.462	138,0	26.441	10,50

Tabel 14 Hasil Hill-climbing per run

Varian	Seed	Objective awal	Objective akhir	Iterasi	Evaluasi	Durasi (s)	Alasan berhenti
Steepest Ascent	1	879	2.243	16	4.271	1,25	local_optimum
Steepest Ascent	2	1.107	2.265	14	3.339	1,03	local_optimum
Steepest Ascent	3	1.039	2.279	18	3.455	1,34	local_optimum
Stochastic	1	879	2.169	29	7.057	2,10	local_optimum
Stochastic	2	1.107	2.126	25	5.982	1,87	local_optimum
Stochastic	3	1.039	2.283	27	5.501	2,02	local_optimum
Sideways Move	1	879	2.406	124	14.656	9,12	max_sideways
Sideways Move	2	1.107	2.403	117	12.920	8,42	max_sideways
Sideways Move	3	1.039	2.376	122	12.466	8,87	max_sideways
Random Restart	1	879	2.297	134	26.226	10,45	max_restart
Random Restart	2	1.107	2.462	134	25.348	10,28	max_restart
Random Restart	3	1.039	2.338	146	27.749	10,77	max_restart

Tabel 15 Sideways Move Hill-climbing: pemakaian sideways move

Seed	Sideways terpakai / maksimum	Iterasi	Alasan berhenti
1	100 / 100	124	max_sideways
2	100 / 100	117	max_sideways
3	100 / 100	122	max_sideways

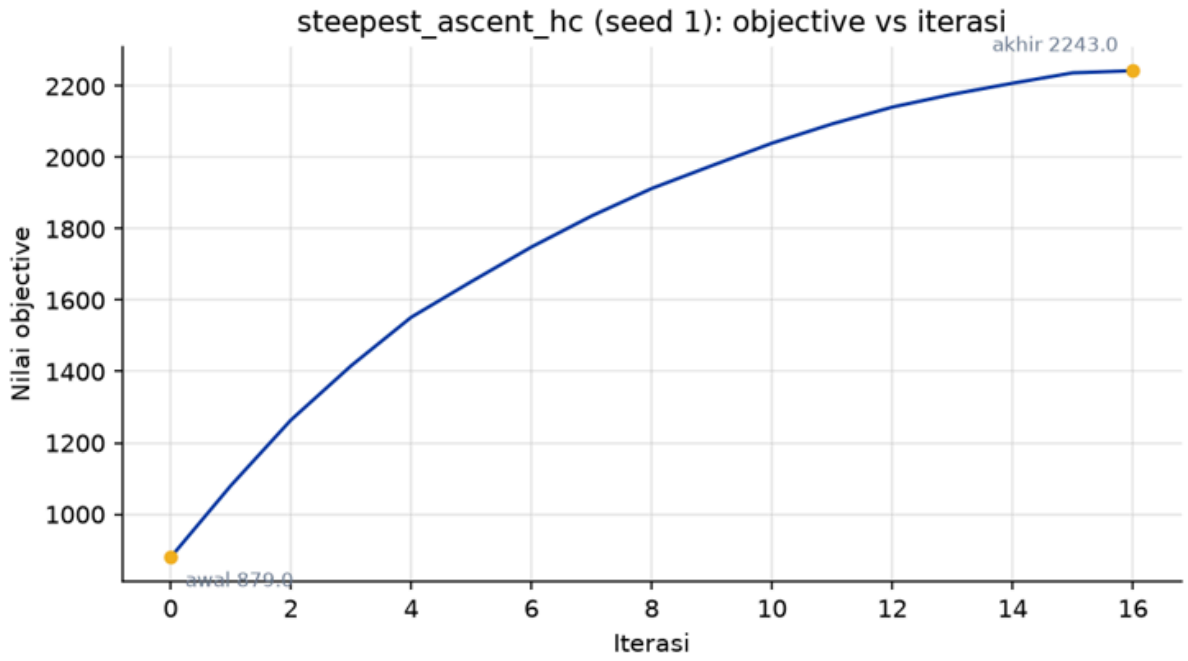
Tabel 16 Random Restart Hill-climbing: restart dan iterasi per restart

Seed	Restart	Iterasi per pendakian (11 pendakian)	Objective akhir per pendakian	Terbaik
1	10 dari maks. 10	16, 14, 10, 11, 9, 11, 13, 9, 12, 13, 16	2243, 2238, 2193, 2088, 2186, 2285, 2249, 2072, 2220, 2279, 2297	2.297 (restart ke-10)
2	10 dari maks. 10	14, 11, 9, 17, 13, 11, 13, 9, 15, 10, 12	2265, 2278, 2098, 2462, 2088, 2338, 2250, 1978, 2225, 2226, 2227	2.462 (restart ke-3)
3	10 dari maks. 10	18, 11, 13, 10, 12, 15, 13, 12, 12, 13, 17	2279, 2240, 2267, 2259, 1944, 2338, 2267, 2082, 2232, 2166, 2127	2.338 (restart ke-5)

Pendakian pertama pada Tabel 16 bukan *restart*, sehingga "*restart* ke-k" adalah pendakian ke-(k + 1).



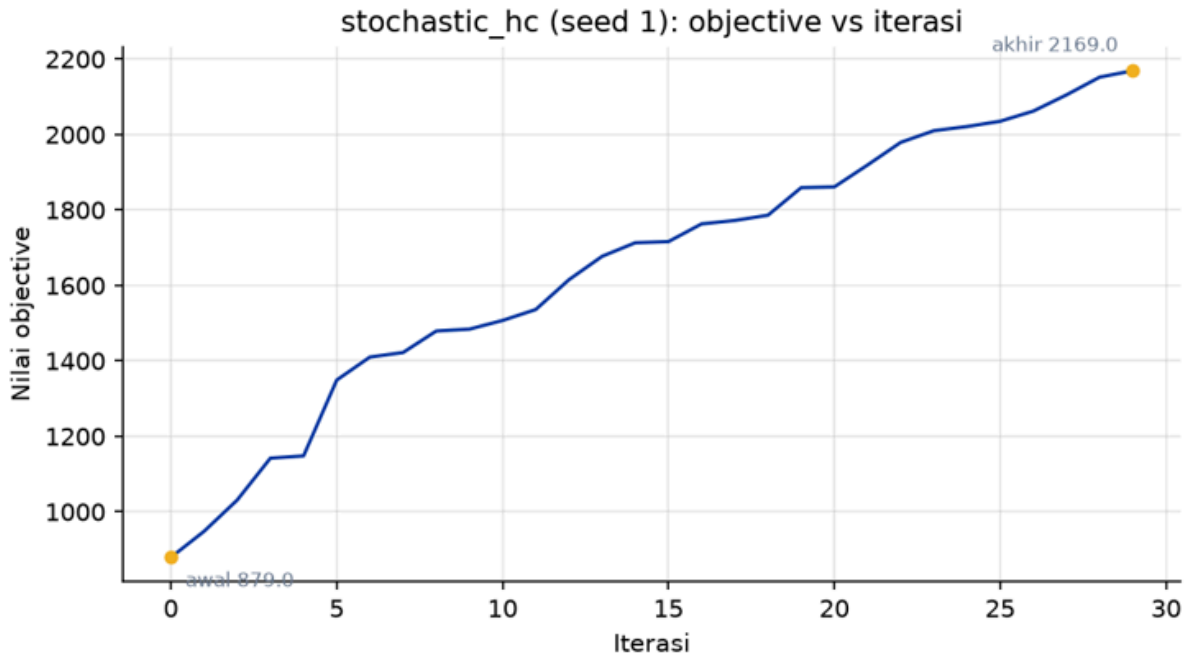
Gambar 2 State awal dan akhir Steepest Ascent HC (seed 1)



Gambar 3 Plot objective terhadap iterasi, Steepest Ascent HC (seed 1)



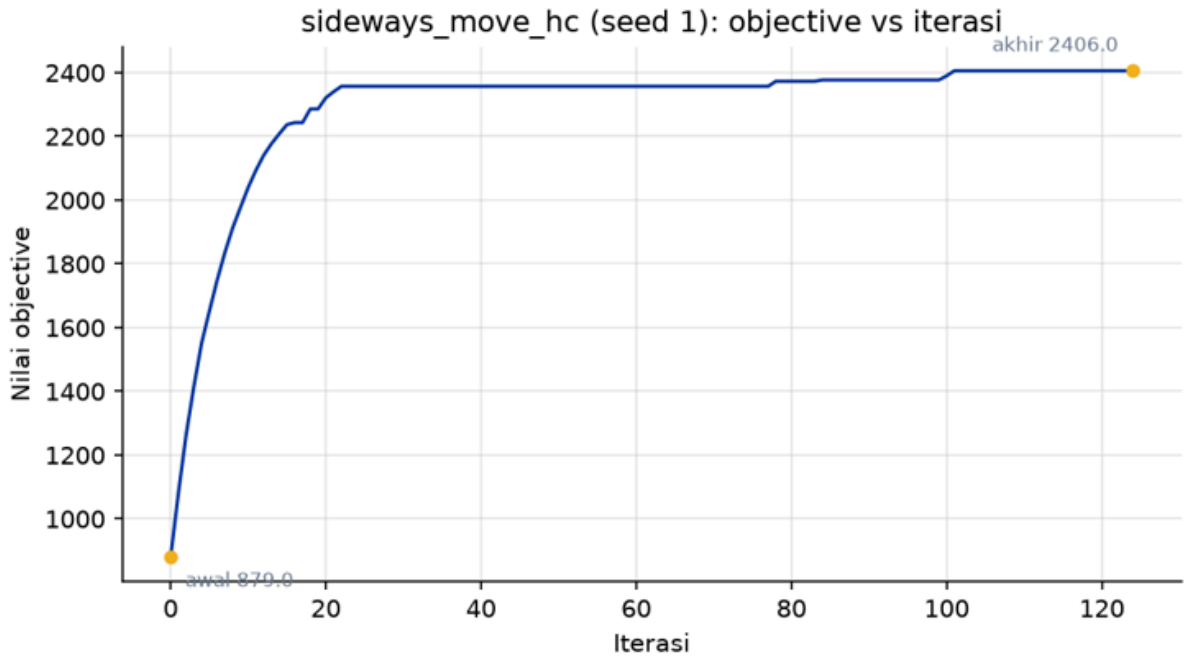
Gambar 4 State awal dan akhir Stochastic HC (seed 1)



Gambar 5 Plot objective terhadap iterasi, Stochastic HC (seed 1)



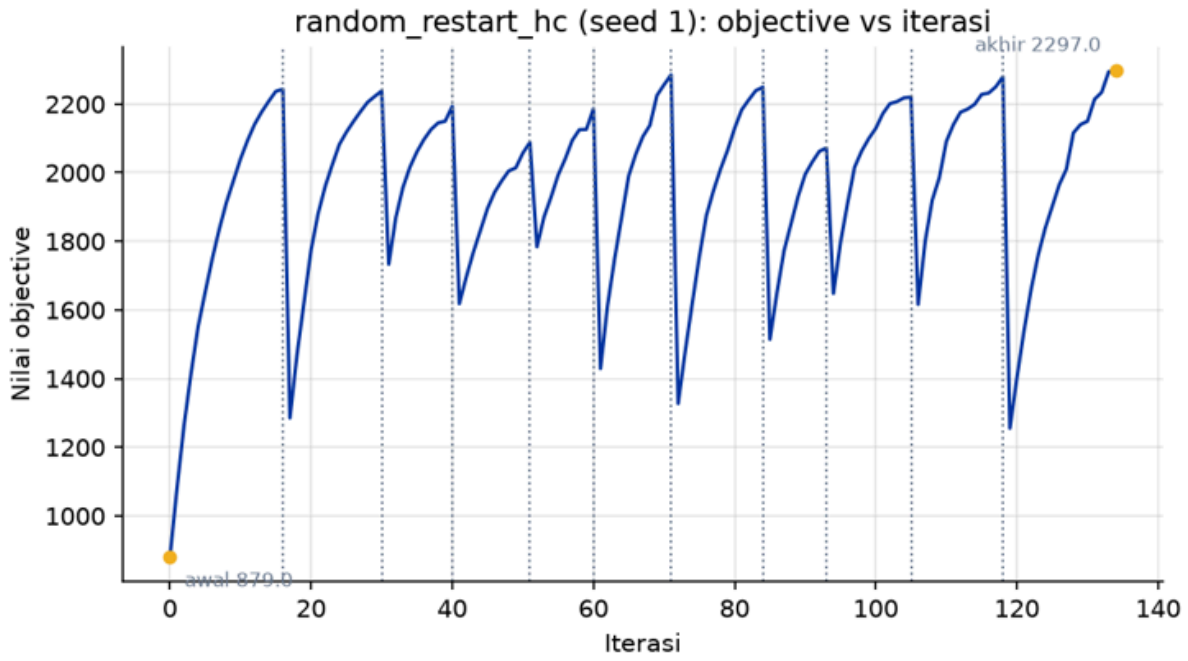
Gambar 6 State awal dan akhir Sideways Move HC (seed 1)



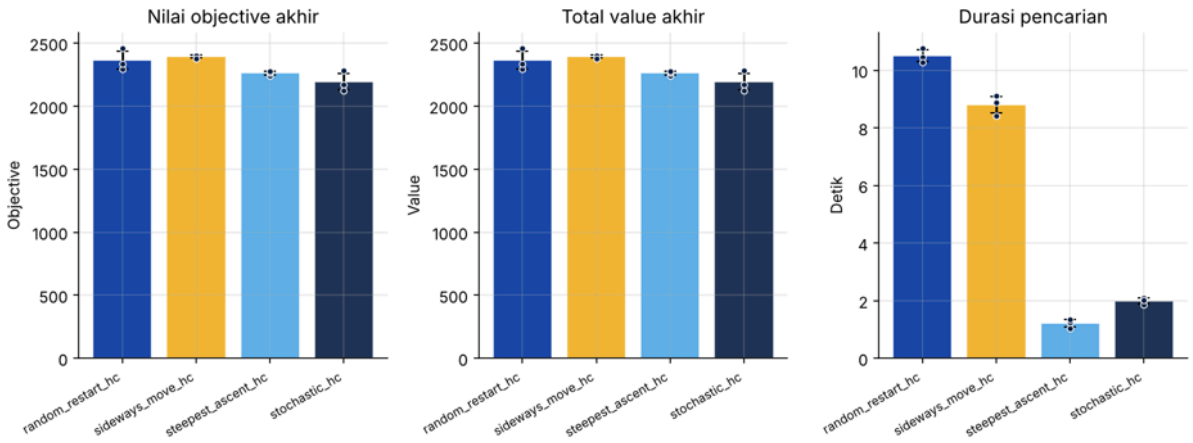
Gambar 7 Plot objective terhadap iterasi, Sideways Move HC (seed 1)



Gambar 8 State awal dan akhir Random Restart HC (seed 1)



Gambar 9 Plot objective terhadap iterasi, Random Restart HC (seed 1)



Gambar 10 Perbandingan hasil varian Hill-climbing pada seluruh run

Analisis

Urutan rata-rata value akhir adalah Sideways Move (2.395,00), Random Restart (2.365,67), Steepest Ascent (2.262,33), dan Stochastic (2.192,67). Steepest Ascent dan Stochastic selalu berhenti karena local optimum setelah 14 sampai 29 langkah. Hasil keduanya ditentukan oleh local optimum terdekat dari state awal sehingga plot objective-nya (Gambar 3 dan Gambar 5) naik pada setiap iterasi lalu berhenti. Pada seed 1, Steepest Ascent mengisi truk dari 12 paket (berat 225 dari 500) menjadi 24 paket (berat 460).

Ketiga run Sideways Move berhenti karena kuota 100 sideways move habis, bukan karena local optimum. Sampai sideways move pertama, lintasannya sama dengan Steepest Ascent, langkah datar pertama terjadi tepat pada local optimum Steepest Ascent untuk seed yang sama (2.243, 2.265, dan 2.279). Setelah itu, 100 langkah tidak mengubah objective sehingga plot pada Gambar 7 didominasi bagian mendatar yang diselingi kenaikan kecil, kenaikan terakhir terjadi pada iterasi ke-101, ke-116, dan ke-66. Ini menunjukkan bahwa persoalan ini memiliki banyak dataran, banyak tetangga bernilai sama, misalnya rotasi paket atau pemindahan paket ke posisi lain yang tidak mengubah total value. Dengan menerima langkah yang setara, pencarian dapat menyeberangi dataran dan menemukan langkah naik berikutnya. Hasilnya 5,9% lebih tinggi daripada Steepest Ascent dengan simpangan baku terkecil di antara varian Hill-climbing (16,52). Biayanya 13.347 evaluasi (3,6 kali Steepest Ascent) dan 8,80 detik (7,3 kali).

Pada Random Restart, pendakian pertama setiap seed menghasilkan nilai yang sama persis dengan Steepest Ascent (2.243, 2.265, dan 2.279), karena keduanya mulai dari state awal dan urutan bilangan acak yang sama. Pendakian berikutnya berakhir pada nilai yang tersebar lebar, dari 1.944 sampai 2.462, dengan 9 sampai 18 iterasi per pendakian. Pendakian terbaik ditemukan pada restart ke-10, ke-3, dan ke-5, dan menaikkan hasil pendakian pertama sebesar 54, 197, dan 59. Rata-ratanya 4,6% lebih tinggi daripada Steepest Ascent, tetapi simpangan bakunya paling besar (85,91), karena hasil akhir bergantung pada apakah salah satu state awal acak jatuh di daerah yang baik. Seed 2 menghasilkan 2.462, nilai tertinggi dari seluruh run Hill-climbing. Biayanya 26.441 evaluasi (7,2 kali Steepest Ascent) dan 10,50 detik (8,7 kali). Garis putus-putus pada Gambar 9 menandai awal setiap restart, dan setiap pendakian terlihat sebagai kurva naik tersendiri.

Stochastic Hill-climbing juga mengevaluasi seluruh neighborhood pada setiap langkah sehingga biaya per langkahnya setara dengan Steepest Ascent. Bedanya, Stochastic memilih sembarang tetangga yang naik, bukan yang terbaik sehingga kenaikan per langkah lebih kecil. Akibatnya membutuhkan lebih banyak langkah (27 dibanding 16) dan lebih banyak evaluasi (6.180 dibanding 3.688), tetapi rata-rata berakhir lebih rendah dan lebih bervariasi (simpangan baku 81,13). Pemilihan acak kadang menguntungkan: pada seed 3, Stochastic mencapai 2.283, sedikit di atas Steepest Ascent (2.279).

Pada semua varian, berat akhir mendekati kapasitas lebih cepat daripada volume: state akhir terbaik Sideways Move seed 1 memakai berat 496 dari 500 tetapi volume 813 dari 1.000, sehingga kapasitas berat adalah constraint yang paling membatasi.

2.12 Hasil Eksperimen Simulated Annealing

Simulated Annealing dijalankan tiga kali dengan *seed* 1, 2, dan 3 pada persoalan medium dengan parameter pada Tabel 11. *State* awal setiap *seed* sama dengan *state* awal Hill-climbing. Ketiga *run* berakhir *feasible*.

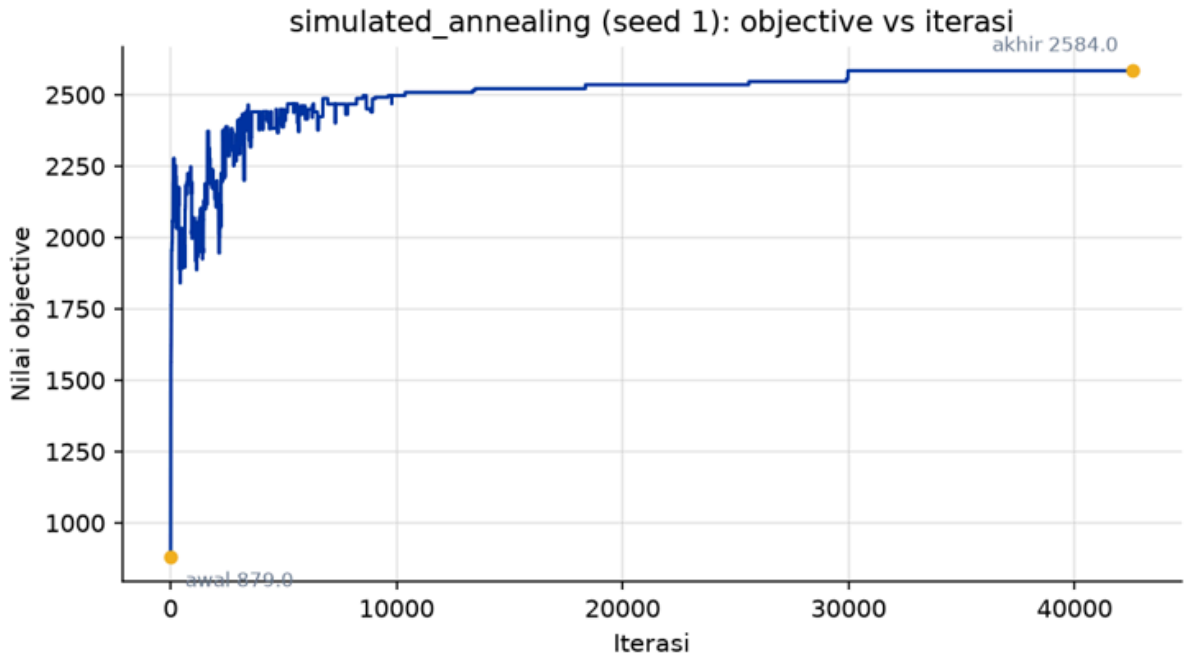
Tabel 17 Hasil Simulated Annealing per run

Seed	Objective awal	Objective akhir	Iterasi	Durasi (s)	Frekuensi stuck	Move diterima	Move memburuk diterima
1	879	2.584	42.582	19,05	65	26.022	133
2	1.107	2.615	42.582	20,34	69	24.144	120
3	1.039	2.615	42.582	19,11	64	25.742	140

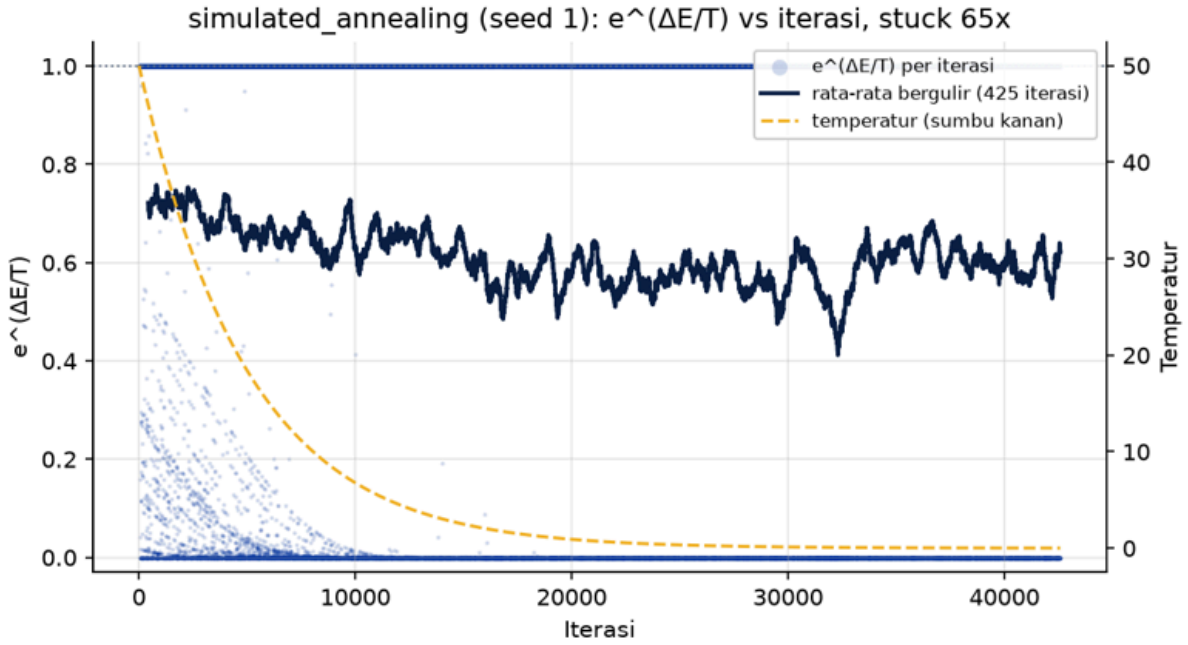
Ketiga run berhenti karena temperatur mencapai t_{min}. Pada ketiganya, objective terbaik yang pernah dikunjungi sama dengan objective akhir.



Gambar 13 State awal dan akhir Simulated Annealing (seed 1)



Gambar 14 Plot objective terhadap iterasi, Simulated Annealing (seed 1)



Gambar 15 Plot $e^{(\Delta E/T)}$ terhadap iterasi, Simulated Annealing (seed 1)

Analisis

Rata-rata value akhir Simulated Annealing adalah 2.604,67 dengan simpangan baku 17,90, atau sekitar 0,7% dari rata-ratanya. Dua dari tiga run berakhir di 2.615, nilai tertinggi dari seluruh run pada persoalan medium. Hasil ini 209,67 lebih tinggi daripada varian Hill-climbing terbaik (Sideways Move). Biayanya juga paling besar, setiap run memakai

42.583 evaluasi (satu untuk state awal dan satu per iterasi), sekitar 3,2 kali Sideways Move, dengan durasi rata-rata 19,50 detik. Pada seed 1, state akhir memuat 28 paket dengan berat 498 dari 500 (Gambar 13).

Dari 42.582 tetangga yang dibangkitkan, 24.144 sampai 26.022 diterima (57% sampai 61%). Namun hanya 120 sampai 140 di antaranya yang merupakan move memburuk, atau sekitar 0,5% dari move yang diterima. Jadi sebagian besar move yang diterima memiliki $\Delta E \geq 0$, dan banyak di antaranya tidak mengubah nilai, misalnya rotasi atau swap paket bernilai sama. Ini sejalan dengan banyaknya dataran yang terlihat pada Sideways Move, dan menjelaskan rata-rata bergulir $e^{(\Delta E/T)}$ sekitar 0,6 pada Gambar 15.

Plot $e^{(\Delta E/T)}$ pada Gambar 15 memperlihatkan dua pita: nilai 1 untuk tetangga yang tidak lebih buruk, dan nilai mendekati 0 untuk tetangga yang jauh lebih buruk. Di antara keduanya terdapat sebaran titik yang hanya muncul ketika temperatur masih tinggi. Sebaran ini menghilang seiring turunnya T: move memburuk terakhir yang diterima terjadi pada iterasi ke-9.784, ke-10.046, dan ke-9.248, ketika T sekitar 6,7 sampai 7,9. Setelah itu Simulated Annealing praktis berperilaku seperti Hill-climbing yang memilih tetangga acak. Plot objective (Gambar 14) sesuai dengan pola ini: nilai naik cepat pada ratusan iterasi pertama, berfluktuasi selama temperatur masih tinggi, lalu naik dalam undakan kecil. Nilai akhir pertama kali dicapai pada iterasi ke-29.971, ke-22.694, dan ke-35.301.

Frekuensi stuck berkisar 64 sampai 69 kejadian per run, atau sekitar satu kejadian setiap 620 sampai 670 iterasi, kejadian pertama tercatat antara iterasi ke-6.707 dan ke-7.232. Artinya setelah sekitar iterasi ke-7.000, pencarian sering melewati 500 iterasi berturut-turut tanpa satu pun langkah naik, baik karena berjalan di dataran maupun karena berada di sekitar local optimum. Meskipun begitu, kemampuan menerima move yang lebih buruk saat temperatur masih tinggi membuat pencarian dapat keluar dari local optimum awal sehingga hasil akhirnya lebih tinggi daripada semua varian Hill-climbing. Karena state terbaik yang pernah dikunjungi sama dengan state akhir pada ketiga run, opsi return_best = true tidak akan mengubah hasil eksperimen ini.

2.13 Hasil Eksperimen Genetic Algorithm

Eksperimen dilakukan dengan dua skema: jumlah populasi sebagai kontrol (30) dengan tiga variasi banyak iterasi, dan banyak iterasi sebagai kontrol (40) dengan tiga variasi

jumlah populasi. Setiap konfigurasi dijalankan tiga kali. Konfigurasi populasi 30 dan iterasi 40 muncul pada kedua skema dan menghasilkan *run* yang identik karena parameter dan *seed*-nya sama. *State* awal yang dicatat adalah individu terbaik populasi awal, sehingga nilainya jauh lebih tinggi daripada *state* awal acak pada algoritma lain dan tidak dapat dibandingkan langsung.

Tabel 18 Variasi banyak iterasi (populasi 30)

Iterasi	Value	Std	Evaluasi	Durasi (detik)
20	2097,7	66,5	630	0,28
40	2140,0	28,7	1.230	0,39
60	2182,3	65,6	1.830	0,67

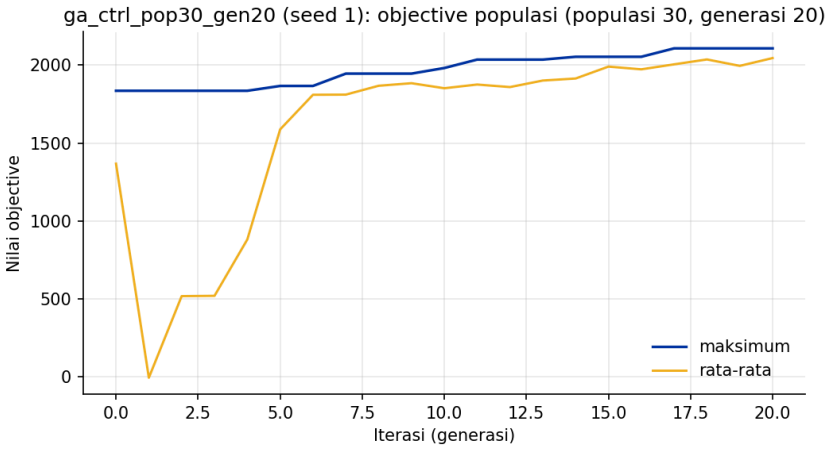
Tabel 19 Variasi jumlah populasi (iterasi 40)

Populasi	Value	Std	Evaluasi	Durasi (detik)
15	1950,0	111,0	615	0,16
30	2140,0	28,7	1.230	0,33
60	2234,0	60,0	2.460	0,72

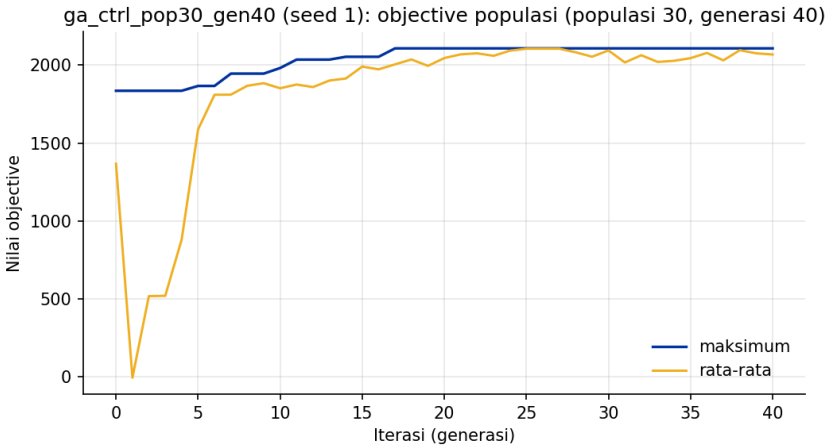
Tabel 20 Hasil Genetic Algorithm per run

Populasi	Iterasi	Seed	Objective awal	Objective akhir	Durasi (detik)
15	40	1	1654	1840	0,15
15	40	2	1751	1948	0,17
15	40	3	1693	2062	0,16
30	40	1	1835	2107	0,33
30	40	2	1964	2154	0,32
30	40	3	1819	2159	0,32
60	40	1	1835	2175	0,64
60	40	2	1964	2295	0,70
60	40	3	1819	2232	0,84
30	20	1	1835	2107	0,30
30	20	2	1964	2027	0,28
30	20	3	1819	2159	0,25
30	40	1	1835	2107	0,49
30	40	2	1964	2154	0,35
30	40	3	1819	2159	0,34
30	60	1	1835	2107	0,65

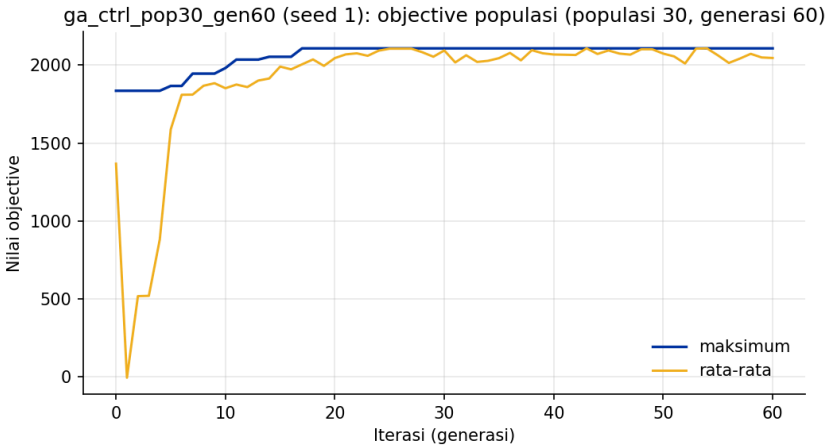
Populasi	Iterasi	Seed	Objective awal	Objective akhir	Durasi (detik)
30	60	2	1964	2227	0,62
30	60	3	1819	2213	0,73



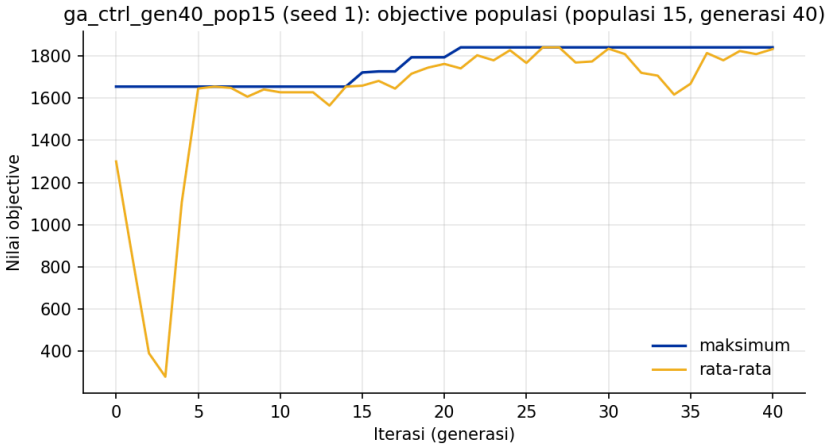
Gambar 16 Plot objective maksimum dan rata-rata populasi (populasi 30, iterasi 20, seed 1)



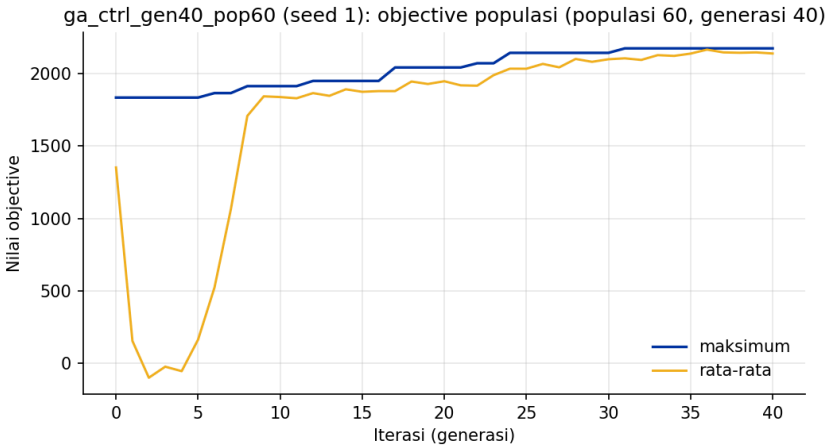
Gambar 17 Plot objective maksimum dan rata-rata populasi (populasi 30, iterasi 40, seed 1)



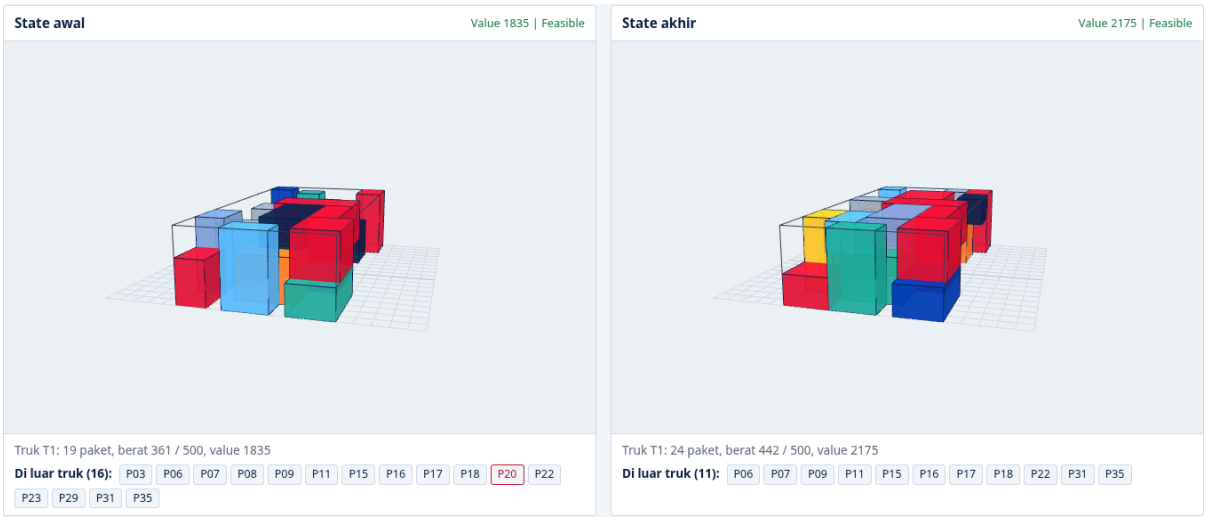
Gambar 18 Plot objective maksimum dan rata-rata populasi (populasi 30, iterasi 60, seed 1)



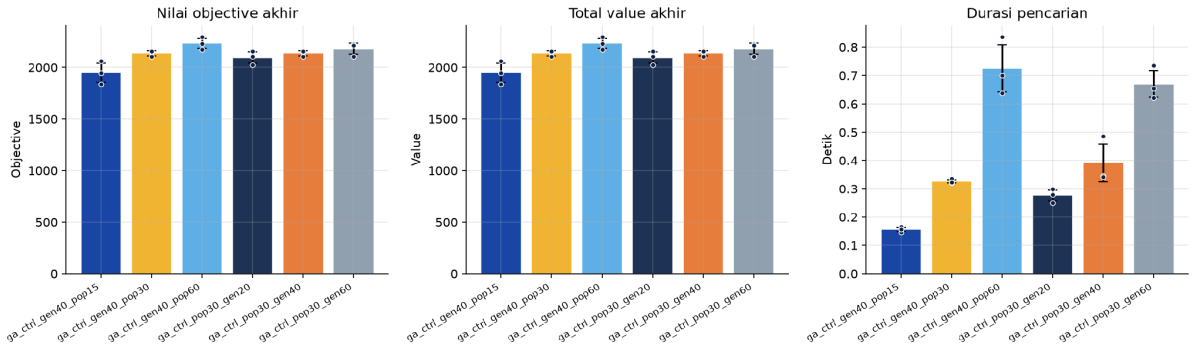
Gambar 19 Plot objective maksimum dan rata-rata populasi (populasi 15, iterasi 40, seed 1)



Gambar 20 Plot objective maksimum dan rata-rata populasi (populasi 60, iterasi 40, seed 1)



Gambar 21 State awal dan akhir Genetic Algorithm (populasi 60, iterasi 40, seed 1)



Gambar 22 Perbandingan hasil seluruh konfigurasi Genetic Algorithm

Analisis

Pengaruh banyak iterasi: pada populasi 30, menaikkan iterasi dari 20 ke 40 lalu 60 menaikkan rata-rata *value* dari 2097,7 ke 2140,0 dan 2182,3 (masing-masing sekitar 2,0%). Kenaikan ini kecil dan tidak konsisten antar-*seed*: pada *seed* 1, hasilnya sama (2.107) pada ketiga banyak iterasi, sehingga perbaikannya terutama berasal dari *seed* 2 dan 3 (Tabel 19). Pada *seed* 1 (Gambar 18), nilai maksimum populasi naik dari 1.835 menjadi 2.107 dalam 17 generasi pertama lalu mendatar. Nilai rata-rata populasi justru turun tajam hingga mendekati nol pada generasi pertama, karena *crossover* seragam antara dua susunan acak banyak menghasilkan anak yang melanggar *constraint* dan dikenai penalti; rata-rata baru pulih mendekati nilai maksimum setelah sekitar 20 generasi, yang menandakan populasi mulai seragam.

Pengaruh jumlah populasi: pada 40 iterasi, menaikkan populasi dari 15 ke 30 lalu 60 menaikkan rata-rata *value* dari 1950,0 ke 2140,0 dan 2234,0 (kenaikan 9,7% dan 4,4%), jauh lebih besar daripada pengaruh iterasi. Pada anggaran evaluasi yang hampir sama, populasi 30 dengan 20 iterasi (630 evaluasi, 2097,7) mengungguli populasi 15 dengan 40 iterasi (615 evaluasi, 1950,0). Dengan demikian, keragaman populasi lebih berharga daripada tambahan generasi pada konfigurasi ini. Durasi bertambah kira-kira sebanding dengan jumlah evaluasi (populasi kali generasi).

Temuan kunci:

- Hasil terbaik diperoleh pada populasi 60 dan iterasi 40 (2234,0) dengan durasi 0,72 detik.
- Menambah jumlah populasi memberi peningkatan lebih besar daripada menambah iterasi, dan pada anggaran evaluasi setara populasi yang lebih besar lebih baik.
- Hasil pada populasi kecil sangat bervariasi (standar deviasi 111,0 pada populasi 15).

- Genetic Algorithm adalah yang tercepat tetapi hasilnya lebih rendah daripada Simulated Annealing dan Hill-climbing yang memakai evaluasi jauh lebih banyak.

2.14 Hasil Eksperimen Bonus

Semua algoritma dijalankan pada persoalan medium dengan *objective* alternatif, dan pada persoalan multi_truck (tiga truk) dengan *objective* bawaan. Genetic Algorithm memakai populasi 30 dan 40 iterasi, dan juga dijalankan dengan *objective* bawaan pada eksperimen `bonus_objective` serta dengan *objective* alternatif pada `bonus_multi_truck`. Parameter algoritma lain sama dengan eksperimen wajib, dan setiap konfigurasi dijalankan tiga kali. Semua *run* bonus berakhir *feasible*. Bonus 2 (seluruh varian Hill-climbing) terpenuhi oleh lima varian pada subbab 2.5 dan 2.11.

Bonus 1: objective function alternatif

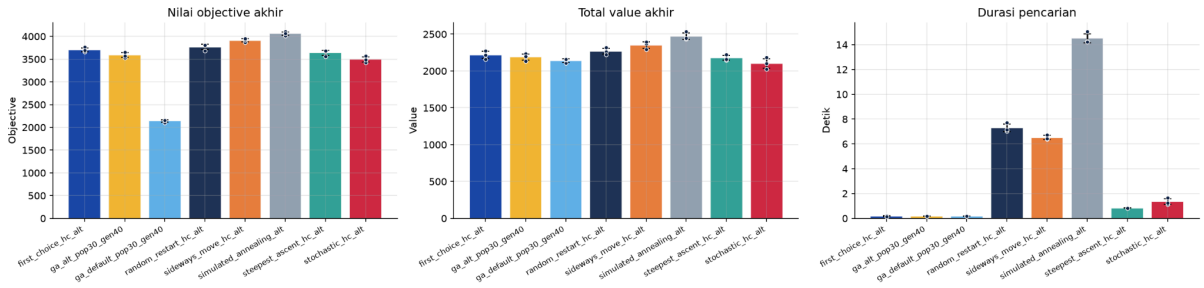
Tabel 21 Total value akhir pada persoalan medium: objective bawaan dan alternatif (3 run)

Algoritma	Objective bawaan	Objective alternatif	Selisih	Selisih (%)
Steepest Ascent	2.262,33	2.175,67	−86,67	−3,8
Stochastic	2.192,67	2.104,00	−88,67	−4,0
Sideways Move	2.395,00	2.352,00	−43,00	−1,8
Random Restart	2.365,67	2.267,33	−98,33	−4,2
Simulated Annealing	2.604,67	2.470,00	−134,67	−5,2
Genetic Algorithm (populasi 30, iterasi 40)	2.140,00	2.188,00	+48,00	+2,2

Tabel 22 Rata-rata ETA paket yang diangkut pada state akhir (medium, 3 run)

Algoritma	Objective bawaan	Objective alternatif
Steepest Ascent	3,81	3,35
Stochastic	4,08	3,42
Sideways Move	3,85	3,46
First-Choice	4,03	3,50

Simulated Annealing	3,95	3,70
Genetic Algorithm (populasi 30, iterasi 40)	3,82	3,70



Gambar 23 Perbandingan hasil seluruh algoritma pada persoalan medium dengan objective alternatif



Gambar 24 State awal dan akhir Simulated Annealing dengan objective alternatif (seed 1)

Objective alternatif mengoptimalkan besaran yang berbeda, yaitu *value* berbobot urgensi ETA ditambah utilisasi, sehingga total *value* murni tidak selalu ikut naik. Pada lima dari tujuh algoritma total *value* sedikit lebih rendah dibanding *objective* bawaan, sedangkan pada First-Choice dan Genetic Algorithm lebih tinggi (Tabel 21). Penurunan pada Simulated Annealing dan Steepest Ascent melebihi simpangan baku antar-*run*, sedangkan perubahan pada algoritma lain berada pada orde satu sampai dua simpangan baku, sehingga dengan tiga *run* arah dampaknya belum konsisten antar-algoritma. Tiga algoritma teratas tetap sama pada kedua *objective*: Simulated Annealing, Sideways Move, lalu Random Restart.

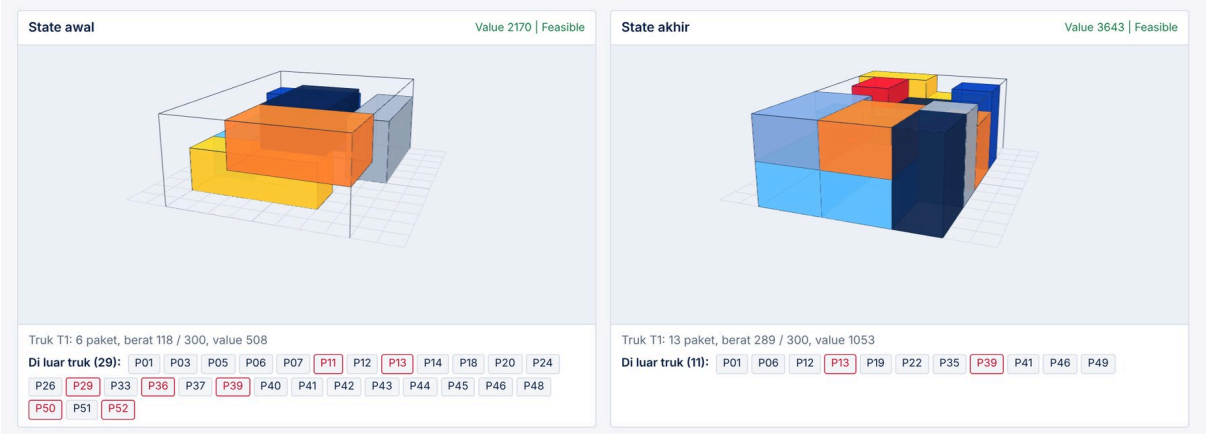
Efek yang dituju *objective* alternatif terlihat jelas pada ETA. Rata-rata ETA seluruh paket persoalan medium adalah 3,91. Dengan *objective* alternatif, rata-rata ETA paket yang diangkut turun pada ketujuh algoritma, misalnya dari 3,81 ke 3,35 pada Steepest Ascent dan dari 3,95 ke 3,70 pada Simulated Annealing (Tabel 22). Artinya pencarian mengganti sebagian paket bernilai tinggi dengan paket yang lebih mendesak, sesuai rancangan *objective*.

Bonus 3: multi-truk

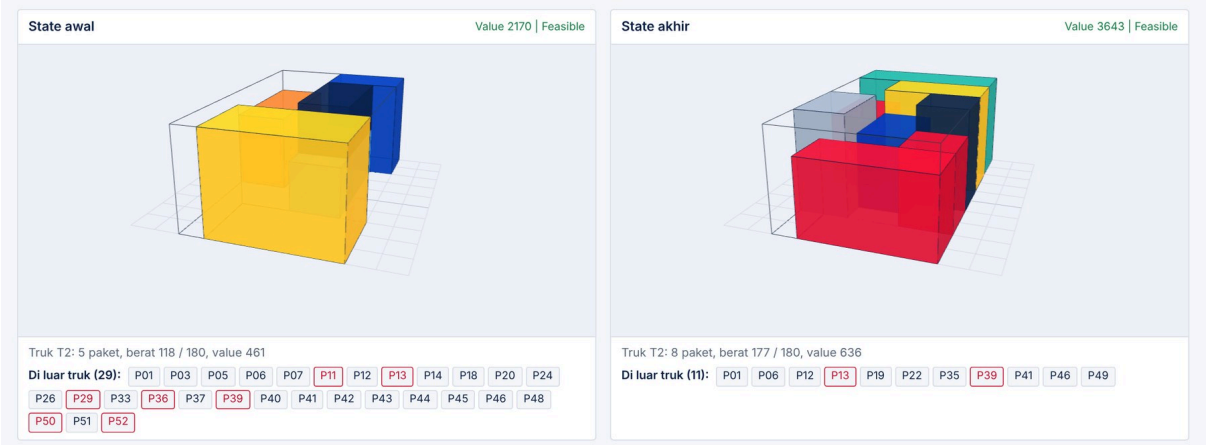
Tabel 23 Total value akhir pada persoalan multi_truck (objective bawaan, rata-rata tiga run)

Algoritma	Truk T1	Truk T2	Truk T3	Total	Std	Evaluasi	Durasi (s)
Steepest Ascent	1.039,33	601,00	1.800,33	3.440,67	41,36	7.899	2,73
Stochastic	995,33	616,67	1.826,67	3.438,67	130,54	18.331	6,33
Sideways Move	1.040,67	665,67	2.003,00	3.709,33	72,15	29.988	16,45
Random Restart	1.048,67	558,00	2.027,67	3.634,33	53,27	78.757	28,22
First-Choice	971,33	626,33	1.887,33	3.485,00	35,51	1.091	0,53
Simulated Annealing	1.032,33	668,33	1.981,00	3.681,67	76,70	42.583	15,10
Genetic Algorithm (populasi 30, iterasi 40)	845,67	573,00	1.807,00	3.225,67	19,55	1.230	0,27

Genetic Algorithm dengan objective alternatif pada persoalan multi_truck menghasilkan total value rata-rata 3.137,00 (simpangan baku 117,66).



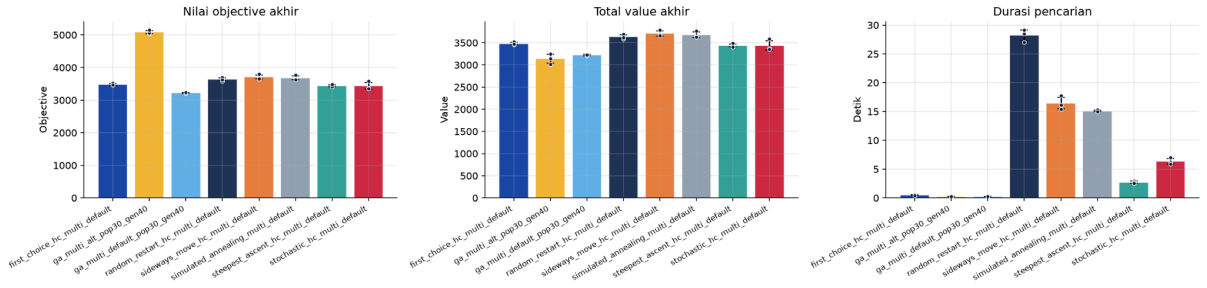
Gambar 25 State awal dan akhir truk T1 pada persoalan multi_truck (Simulated Annealing, seed 1)



Gambar 26 State awal dan akhir truk T2 pada persoalan multi_truck (Simulated Annealing, seed 1)



Gambar 27 State awal dan akhir truk T3 pada persoalan multi_truck (Simulated Annealing, seed 1)



Gambar 28 Perbandingan hasil seluruh algoritma pada persoalan multi_truck

Total *value* tertinggi pada persoalan multi-truk dicapai Sideways Move (3.709,33), disusul Simulated Annealing (3.681,67) dan Random Restart (3.634,33), sedangkan Genetic Algorithm terendah (3.225,67). Selisih Sideways Move dan Simulated Annealing (27,67) lebih kecil daripada simpangan baku keduanya (72,15 dan 76,70), sehingga keduanya setara. Hasil ini berbeda dengan persoalan satu truk yang jelas dimenangkan Simulated Annealing. Satu dugaan kami adalah ruang pencarian yang lebih besar (52 paket, tiga truk, rata-rata 601 tetangga) membuat 42.582 iterasi Simulated Annealing relatif lebih sedikit, tetapi dugaan ini belum diuji.

Truk T3, yang terbesar, mengangkut 52% sampai 56% total *value* pada semua algoritma (Tabel 23). Gambar 25 sampai Gambar 27 memperlihatkan bahwa pada *state* akhir Simulated Annealing *seed* 1 setiap truk terisi jauh lebih padat daripada *state* awal: T1 dari 6 menjadi 13 paket, T2 dari 5 menjadi 8 paket, dan T3 dari 12 menjadi 20 paket. Berat akhirnya 289 dari 300, 177 dari 180, dan 449 dari 450, sedangkan volume akhirnya 319 dari 384, 198

dari 240, dan 592 dari 800. Seperti pada persoalan satu truk, kapasitas berat adalah *constraint* yang paling membatasi.

2.15 Analisis Perbandingan Antaralgoritma

Batas atas

Nilai optimum global tidak diketahui, sehingga kedekatan hasil dinilai terhadap batas atas yang diperoleh dengan relaksasi. Kami menghitung tiga relaksasi yang mengabaikan bentuk geometri paket, yaitu *knapsack* pecahan (*fractional knapsack*) dengan kapasitas berat saja, *knapsack* pecahan dengan volume truk saja, dan *knapsack* 0/1 yang memakai kapasitas berat dan volume sekaligus, diselesaikan dengan *dynamic programming*. Ketiganya adalah batas atas yang sah, dan yang paling ketat adalah *knapsack* 0/1, dengan 2.683 untuk medium dan 4.049 untuk multi_truck. Pada multi_truck, relaksasi memakai kapasitas dan volume gabungan ketiga truk. Batas ini masih longgar karena mengabaikan dimensi, *overlap*, penyangga, dan paket pecah-belah, sehingga optimum sebenarnya lebih rendah dan jarak hasil ke optimum sebenarnya lebih kecil daripada yang tampak.

Tabel 24 Batas atas dan nilai terbaik yang ditemukan

Persoalan	Total value seluruh paket	Pecahan, berat	Pecahan, volume	0/1, berat dan volume	Terbaik ditemukan
medium	3.217	2.693,1	2.903,4	2.683	2.615 (97,5%)
multi_truck	4.829	4.056,9	4.369,3	4.049	3.792 (93,7%)

Tabel 25 Perbandingan algoritma pada persoalan medium (objective bawaan, tiga run)

Algoritma	Value rata-rata	% batas atas	Std	Std / rata-rata (%)	Evaluasi	Durasi (s)
Simulated Annealing	2.604,67	97,1	17,90	0,7	42.583	19,50
Sideways Move HC	2.395,00	89,3	16,52	0,7	13.347	8,80
Random Restart HC	2.365,67	88,2	85,91	3,6	26.441	10,50
Steepest Ascent HC	2.262,33	84,3	18,15	0,8	3.688	1,21
Genetic Algorithm (populasi 60, iterasi 40)	2.234,00	83,3	60,02	2,7	2.460	0,72
Stochastic HC	2.192,67	81,7	81,13	3,7	6.180	1,99

Jawaban pertanyaan analisis

1. Seberapa dekat setiap algoritma dengan global optima, dan mengapa? Simulated Annealing paling dekat dengan rata-rata 97,1% dari batas atas, dan hasil terbaik 2.615 (97,5%) sehingga jaraknya ke optimum sebenarnya paling banyak 2,9%. Sideways Move dan Random Restart mencapai 89,3% dan 88,2%, sedangkan Steepest Ascent, Genetic Algorithm, Stochastic, dan First-Choice berada pada 78,3% sampai 84,3%. Hill-climbing hanya menerima perbaikan (atau langkah setara pada Sideways Move) sehingga berhenti di local optimum atau dataran terdekat. Simulated Annealing dapat menerima move yang lebih buruk saat temperatur tinggi sehingga keluar dari local optimum awal, dan juga memakai evaluasi paling banyak. Genetic Algorithm hanya memakai ratusan sampai ribuan evaluasi, dan crossover seragam per paket sering merusak susunan yang sudah baik.
2. Bagaimana perbandingan hasil antaralgoritma? Urutan rata-rata value adalah Simulated Annealing, Sideways Move, Random Restart, Steepest Ascent, Genetic Algorithm populasi 60, dan Stochastic. Keunggulan Simulated Annealing atas Sideways Move (sekitar 210) jauh lebih besar daripada simpangan baku antar-run. Sebaliknya, selisih Random Restart dan Sideways Move (29) serta selisih Steepest Ascent dan Genetic Algorithm populasi 60 (28) lebih kecil daripada simpangan bakunya sehingga belum dapat dikatakan berbeda dengan tiga run.
3. Bagaimana perbandingan durasinya? Durasi mengikuti banyak evaluasi, yaitu Simulated Annealing paling lama (19,50 detik untuk 42.583 evaluasi), disusul Random Restart (10,50), Sideways Move (8,80), Stochastic (1,99), dan Steepest Ascent (1,21), dan Genetic Algorithm (0,16 sampai 0,72) paling cepat. Satu evaluasi beserta pembangkitan tetangganya memerlukan sekitar 0,3 sampai 0,7 milidetik.
4. Seberapa konsisten hasil akhirnya? Simpangan baku terkecil dimiliki Sideways Move (16,52), Simulated Annealing (17,90), dan Steepest Ascent (18,15), atau sekitar 0,7% sampai 0,8% dari rata-rata. Random Restart, Stochastic, dan Genetic Algorithm populasi 15 paling bervariasi (5,7%). Karena setiap konfigurasi hanya dijalankan tiga kali, ukuran ini bersifat indikatif.
5. Bagaimana pengaruh banyak iterasi dan jumlah populasi pada Genetic Algorithm? Menaikkan populasi dari 15 ke 60 menaikkan value 14,6%, sedangkan menaikkan iterasi dari 20 ke 60 hanya 4,0%. Pada anggaran evaluasi setara, populasi yang lebih besar lebih baik (subbab 2.13).
6. Bagaimana dampak heuristik, objective alternatif, dan multi-truk? Heuristik titik kandidat membuat hampir semua tetangga valid (Tabel 6), sehingga evaluasi tidak

terbuang pada state yang pasti buruk dan neighborhood tetap terkendali (sekitar 265 tetangga pada medium). Penalti memungkinkan move bebas menjelajahi state tidak valid tanpa pernah menjadi menguntungkan. Objective alternatif mengubah total value hanya sedikit tetapi menurunkan rata-rata ETA paket yang diangkut pada semua algoritma, dan multi-truk berjalan pada semua algoritma dengan hasil terbaik 3.792 atau 93,7% dari batas atas armada.

BAB III KESIMPULAN DAN SARAN

3.1 Kesimpulan

Keenam algoritma, yaitu empat varian Hill-climbing, Simulated Annealing, dan Genetic Algorithm, berhasil diimplementasikan dan menghasilkan solusi *feasible* pada semua *run*, termasuk pada persoalan multi-truk dan dengan *objective* alternatif. Representasi berbasis titik kandidat dan *gravity settle* membuat hampir semua tetangga valid, sedangkan penalti tetap memberi arah ketika pencarian melewati *state* tidak valid.

Pada persoalan medium dengan *objective* bawaan, Simulated Annealing memberikan hasil terbaik, dengan rata-rata *value* 2.604,67 dan simpangan baku 17,90. Keunggulan ini dibayar dengan biaya terbesar, yaitu 42.583 evaluasi dan rata-rata 19,50 detik per *run*.

Di antara varian Hill-climbing, mekanisme untuk keluar dari *local optimum* membantu. Sideways Move dan Random Restart menaikkan hasil Steepest Ascent sebesar 5,9% dan 4,6%, dengan durasi 7,3 dan 8,7 kali lebih lama. Semua *run* Sideways Move berhenti karena kuota *sideways move* habis, yang menunjukkan banyaknya dataran pada persoalan ini. Varian yang memilih tetangga secara acak berakhir lebih rendah dan lebih bervariasi.

Pada Genetic Algorithm, jumlah populasi berpengaruh lebih besar daripada banyak iterasi. Menaikkan populasi dari 15 ke 60 menaikkan *value* sebesar 14,6%, sedangkan menaikkan iterasi dari 20 ke 60 hanya 4,0%. Konfigurasi terbaiknya (populasi 60, iterasi 40) mencapai 2.234,00, masih di bawah Sideways Move, Random Restart, dan Simulated Annealing, tetapi hanya memakai 2.460 evaluasi.

Pada persoalan multi-truk, Sideways Move (3.709,33) dan Simulated Annealing (3.681,67) berselisih 27,67, lebih kecil daripada simpangan baku keduanya (72,15 dan 76,70), sehingga dengan tiga *run* belum dapat ditentukan mana yang lebih baik. Dengan *objective* alternatif, Simulated Annealing tetap menghasilkan *value* murni tertinggi (2.470,00), dan rata-rata ETA paket yang diangkut turun pada semua algoritma, sesuai tujuan *objective* tersebut. Pada semua persoalan, kapasitas berat adalah *constraint* yang paling membatasi.

3.2 Saran

- Menjalankan eksperimen dengan anggaran evaluasi yang setara antaralgoritma dan dengan lebih banyak *seed*. Anggaran evaluasi saat ini berkisar dari 501 (First-Choice)

sampai 42.583 (Simulated Annealing), dan dengan tiga *run* beberapa selisih, seperti pada persoalan multi-truk, belum dapat dibedakan dari variasi antar-*run*.

- Mencoba `max_sideways` yang lebih besar, karena semua *run* Sideways Move berhenti akibat kuota habis, bukan karena tidak ada langkah lagi. Random Restart juga dapat memakai Sideways Move sebagai pendaki lewat parameter `base_algorithm` tanpa perubahan kode.
- Menguji jadwal pendinginan Simulated Annealing yang lebih lambat, terutama untuk persoalan yang lebih besar seperti multi_truck, karena uji coba menunjukkan pendinginan lebih lambat menaikkan hasil, dengan konsekuensi durasi yang lebih lama.
- Memperbaiki *crossover* Genetic Algorithm agar menjaga susunan yang sudah baik, misalnya mewarisi seluruh isi satu truk atau satu wilayah ruang dari orang tua yang sama, atau menambahkan langkah perbaikan setelah *crossover*. Sebelum itu, memperbesar populasi lebih bermanfaat daripada menambah iterasi.
- Menghitung batas atas yang memperhitungkan geometri paket, serta mempercepat evaluasi dengan perhitungan selisih (*delta evaluation*) agar persoalan large dapat dijalankan dengan anggaran yang lebih besar.

BAB IV PEMBAGIAN TUGAS

REFERENSI

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=oGTa9kGE3P8 [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8 [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=aI41TGo9J9c [Accessed: Sep. 16, 2026].

[6] T. G. Crainic, G. Perboli, and R. Tadei, "Extreme Point-Based Heuristics for Three-Dimensional Bin Packing," INFORMS Journal on Computing, vol. 20, no. 3, pp. 368–384, 2008.

[7] three.js authors, "three.js: JavaScript 3D Library," MIT License. [Online]. Available: https://threejs.org

LAMPIRAN A FORM PENGGUNAAN GENERATIVE AI

Dengan ini, kami:
Nama/NIM :

1. Edward David Rumahorbo
2. Miguel Rangga Deardo Sinaga
3. Reysha Syafitri Mulya Ramadhan

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Intelegensi Artifisial
Judul Tugas/Proposal : PROGRAM PENCARI KONFIGURASI PENGISIAN PAKET PALING OPTIMAL UNTUK SEBUAH TRUK PETI KEMAS

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan: pendefinisian *workflow* pemrograman, pembagian tugas, dan *frontend* atau visualisasi	Ya	Sedang

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Edward David Rumahorbo
13524036



Miguel Rangga D.S.
13524069



Reysha Syafitri MR
13524137